CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Design Description

for

ELEMENTOPIA

Submitted by:

2526-sem2-it332-53

Ocampo, Danielle Maxine P.
Benolirao, Clyde C.
Ceniza, Jian Marc
Dabalos, Erica Y.
Dibdib, Wayne Kenji B.

Submitted to:

Engr. Ralph P. Laviste



Danielle Maxine P. Ocampo



Erica Y. Dabalos



Jian Marc Ceniza



Clyde C. Benolirao



Wayne Kenji B. Dibdib

Date	Version	Description
May 15, 2026	1.0	Initial creation of SDD document.
May 17, 2026	1.1	Creating the introduction and overall description sections.
May 20, 2026	1.2	Creating the diagrams.

This Software Design Document (SDD) outlines the architectural and component-level design of **Elementopia**, a gamified, web-based educational platform aimed at enhancing chemistry learning for high school students. It is submitted by Group 53 of IT332 under the College of Computer Studies at Cebu Institute of Technology – University as part of the academic requirements for the course.

The SDD details the system's overall structure, including user interface design, backend architecture, database schema, and integration strategies between components. The goal is to provide a clear blueprint for development, implementation, and future maintenance. All designs presented here are based on the specifications defined in the Software Requirements Specification (SRS) and have been adapted to reflect the team’s actual tools, workflows, and constraints.

This document is intended for instructors, project reviewers, and future developers who may build upon or maintain the system.

1. Introduction **6**
1.1. Purpose 6
1.2. Scope 6
1.3. Definitions and Acronyms 6
1.4. References 8
2. Architectural Design **9**
3. Detailed Design **10**
Module 1: Elemental Resonance Puzzle Module *10*
Module 2: Assessment & Progress Module *24*
Module 3: Opponent-Based Challenge Module 29

1.1. Purpose

Elementopia is a **gamified chemistry learning system** for high school students. It enables students to learn chemistry concepts by **combining elements to create compounds**, reinforcing their understanding of **chemical reactions, element properties, and periodic table relationships** in an interactive way.

1.2. Scope

Elementopia is an interactive web application designed for high school chemistry students. It integrates game mechanics to enhance learning, covering topics such as atomic structures, bonding, periodic table interactions, and simulations.

Target Users: High school students learning basic chemistry.

- **Core Features**:
    - **Elemental Resonance Domain Puzzle Rooms** — Students synthesize chemical compounds to trigger environmental reactions that clear physical obstacles blocking their progression.
    - **Meaningful Byproduct Diagnostics** — Replaces binary "Wrong" feedback with contextual chemistry-based guidance for incorrect compound combinations.
    - **Hazmat Protocol Failsafe** — Disables irrelevant elements after 5 failed attempts within 15 seconds to reduce random guessing.
    - **Randomized Compound Selection** — Compounds are drawn from a predefined pool per session to reduce repetitive memorization.
    - **Automated Session Performance Logging** — Tracks compound accuracy and completion speed to Supabase within under 2 seconds per session.
    - **Visual Mastery Dashboard** — Enables students to independently identify knowledge gaps without teacher intervention.
    - **Opponent-Based Challenge Module** — Peer-to-peer competitive matchmaking via auto-generated 5-digit access codes with connection time under 5 seconds.
    - **Speed-to-Compound Evaluation** — Real-time match winner determination based on reaction path efficiency via Supabase Realtime.

○ **Temporary Session Nicknames** — Frictionless onboarding with no user registration required.

○ **Device-Agnostic Session Persistence** — Performance history is accessible across sessions as long as the same nickname is used.

- **Technology Stack**:
  - **Frontend**: React.js (with Axios for API requests).
  - **Backend**: Spring Boot (Java).
  - **Database**: Supabase

1.3. Definitions and Acronyms

- **SRS**: Software Requirements Specification
- **SDD**: Software Design Description
- **UI/UX**: User Interface/User Experience
- **PostgreSQL**: Relational Database Management System
- **API**: Application Programming Interface
- **JWT**: JSON Web Token
- **CRUD**: Create, Read, Update, Delete (for database operations)
- **Supabase:** An open-source Backend-as-a-Service (BaaS) platform that provides database, authentication, storage, and API services for applications.

1.4. References

System Overview

Elementopia follows a three-tier architecture:

Frontend: Developed using React.js and Vite.

Backend: Built with Spring Boot (Java) exposing RESTful APIs.

Database: Supabase (PostgreSQL) for persistent storage and real-time synchronization.

Technology Stack

- **Frontend:** React.js, Vite, Material UI
- **Backend:** Spring Boot, Hibernate (JPA)
- **Database:** Supabase (PostgreSQL)
- **Communication:** REST API, Supabase Realtime for real-time updates

Block Diagram







Frontend

React.js

Backend

Spring Boot

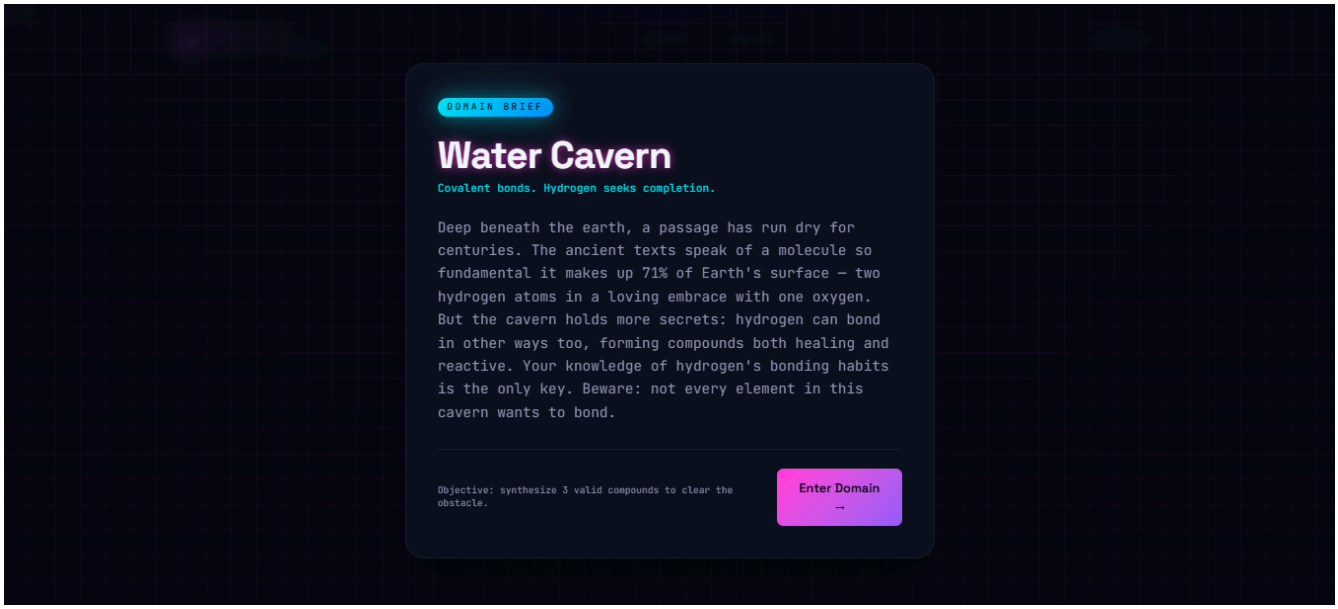
Database

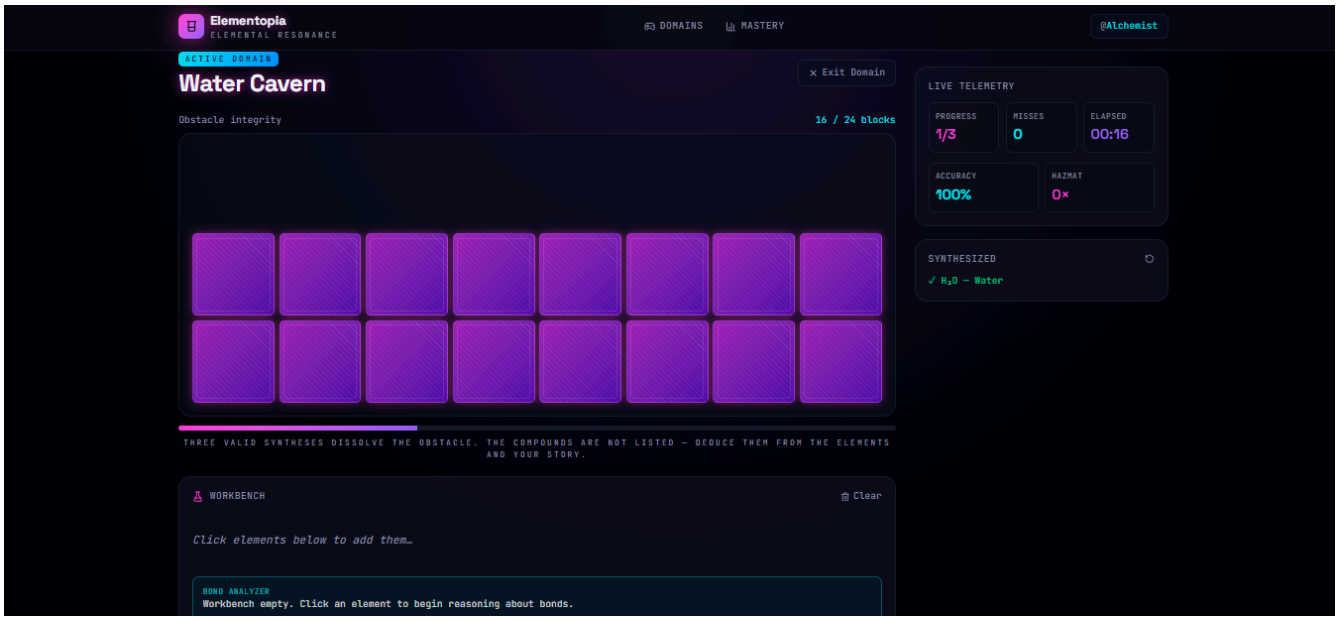
Supabase(PostgreSQL)

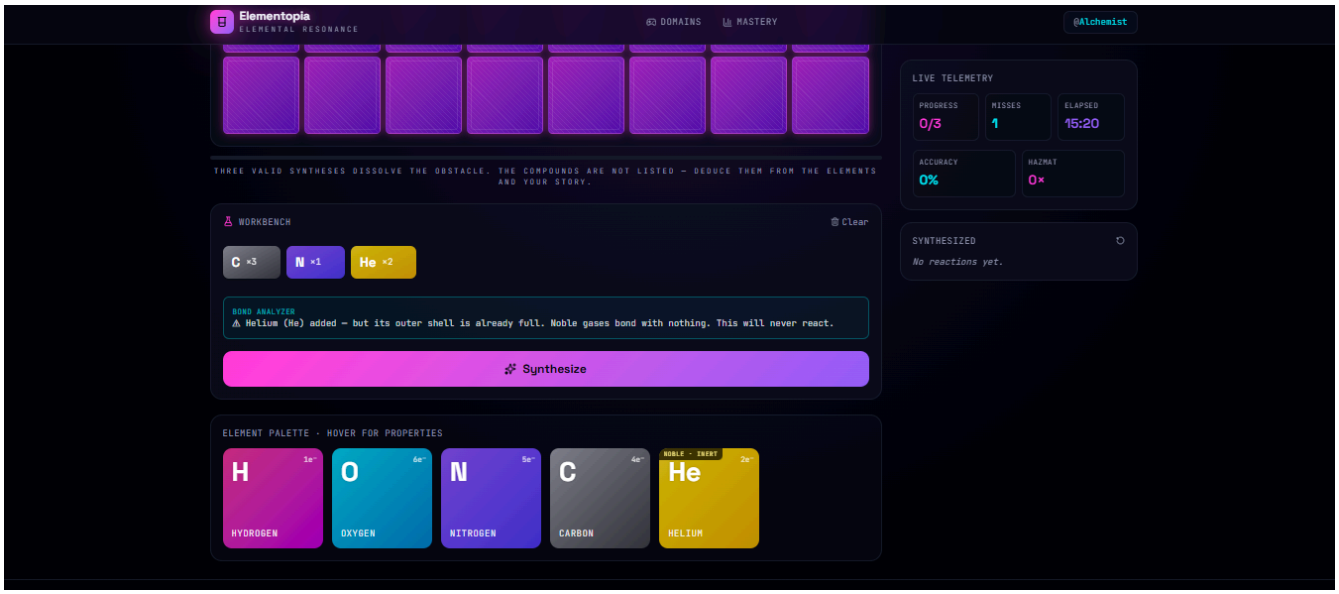
Module 1: Elemental Resonance Puzzle Module

1.1 Elemental Resonance Domain Interaction

- User Interface Design





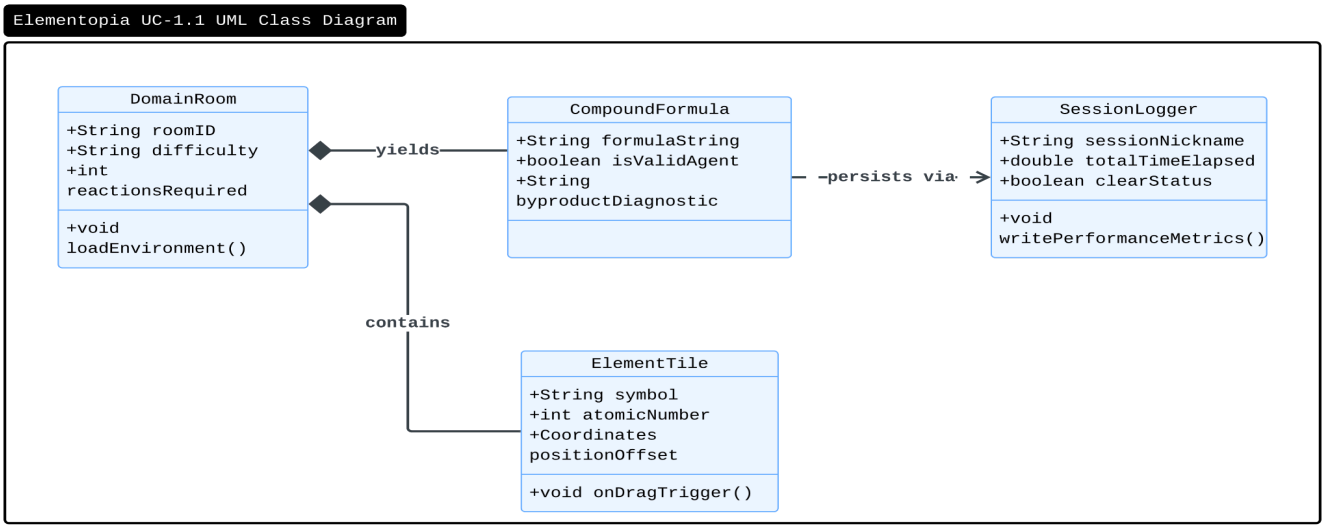


- Front-end component(s)
  - Component Name: ElementalDomainCanvas.jsx
    - Description and purpose: Inherits the core canvas rendering architecture of the original codebase (similar to ChemistrySimulation.jsx). It displays the graphical Domain environment, tracks drag-and-drop boundary interactions , and handles structural layout configurations.
    - Component type or format: React.js component with dynamic context hooks for touch-based tracking and visual asset state management.
  - Component Name: ActiveInventoryTray.jsx
    - Description and purpose: Renders the grid of chemical element tiles provided for the current challenge session. It hooks into state variables to visually disable or alter the interactable inventory selections when downstream processes require it.
    - Component type or format: Functional React.js component fed directly by active gameplay state vectors.

- Back-end component(s)
  - Component Name: ReactionController.java
    - Description and purpose: Exposes dedicated REST endpoints that accept element drop sequences from the client and maps inbound validation operations.
    - Component type or format: Spring Boot @RestController.
  - Component Name:ValidationService.java
    - Description and purpose: Evaluates the compound formula synthesis data sent by the client interface, validating the inputs against curriculum rules rather than traditional right/wrong text queries.

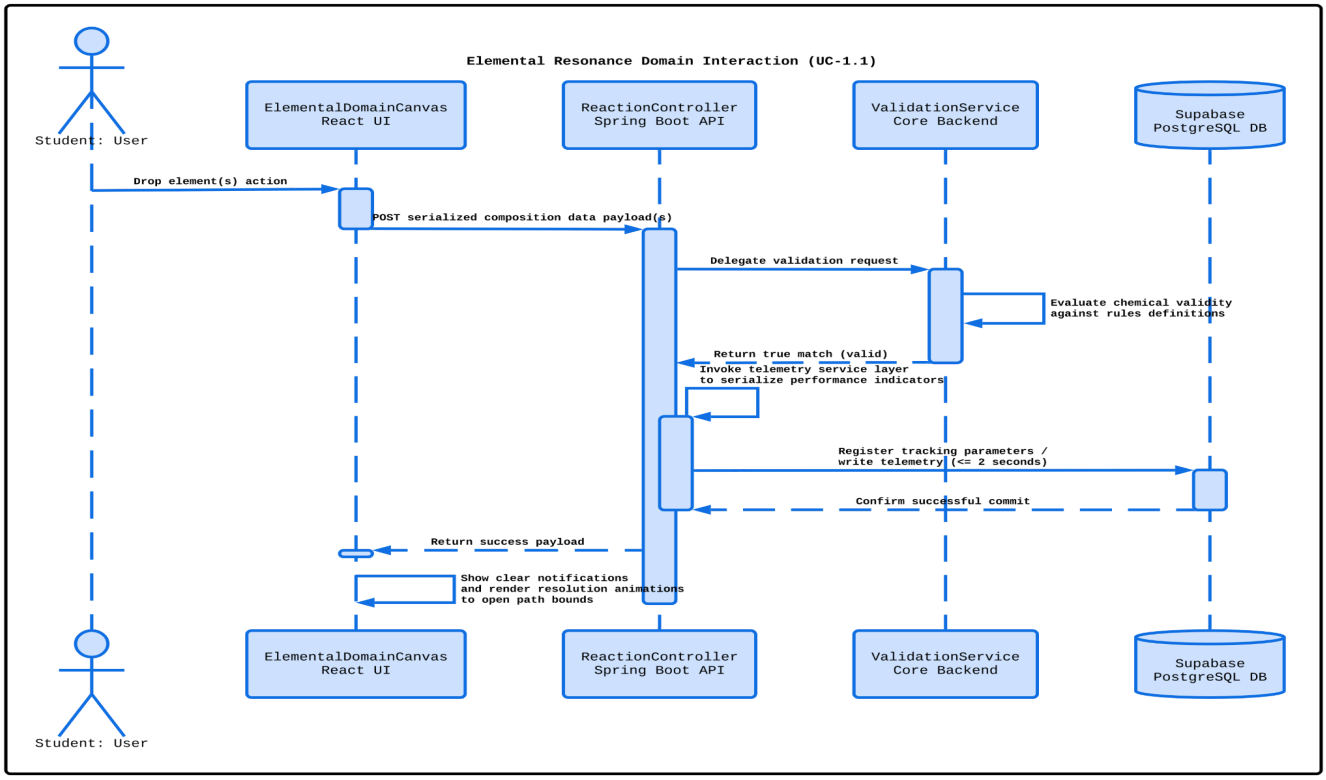
        - Component type or format: Spring Boot service component executing core business tier rules.
    - Component Name: TelemetryService.java
        - Description and purpose: Automatically captures completion speed and attempted variables from the session, routing structural transactions straight to cloud data collections.
        - Component type or format: Spring Boot service tracking database mutations via integrated JPA layers.

- Object-Oriented Components
    - Class Diagram

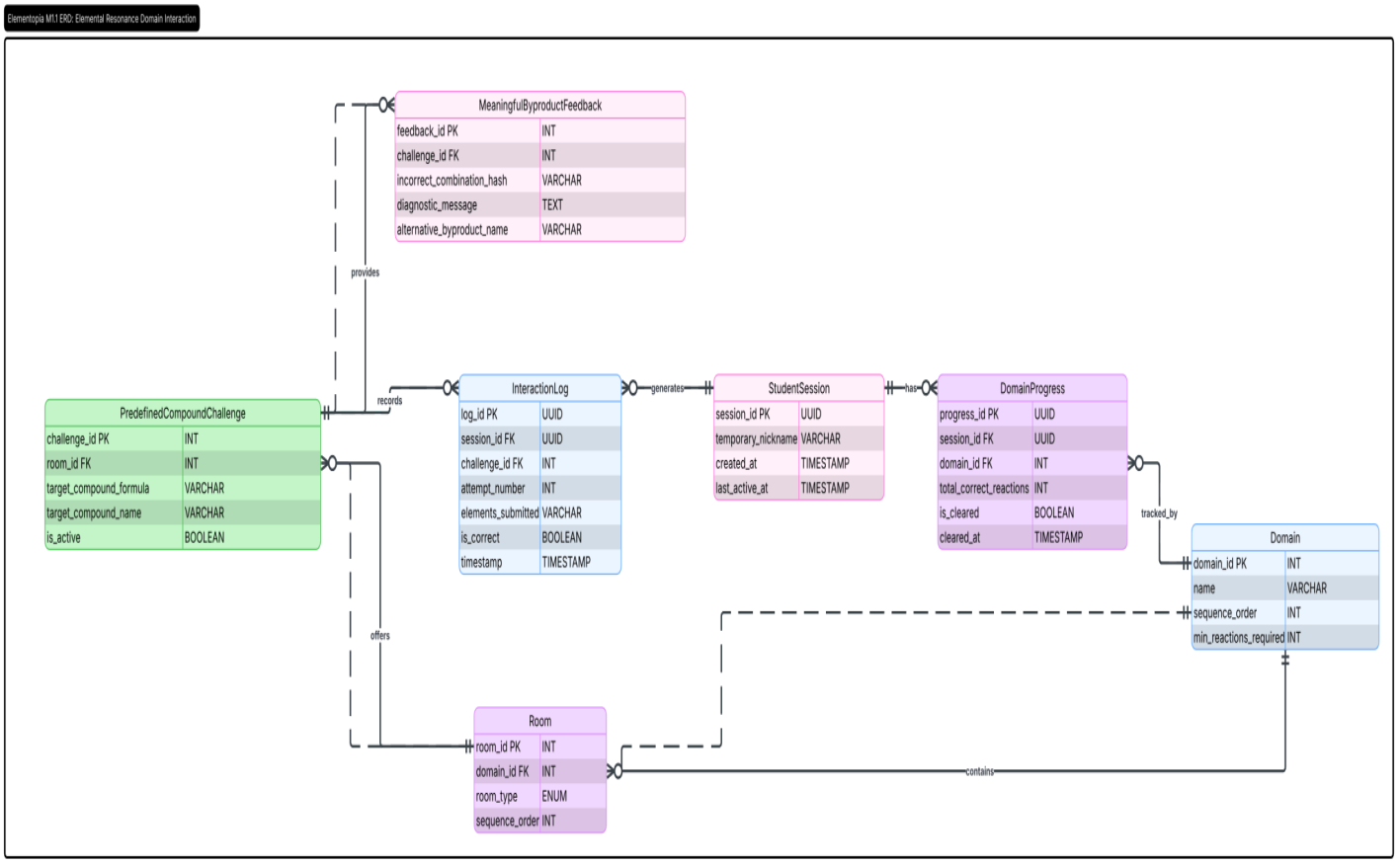


    - Sequence Diagram:

Elemental Resonance Domain Interaction (UC-1.1)

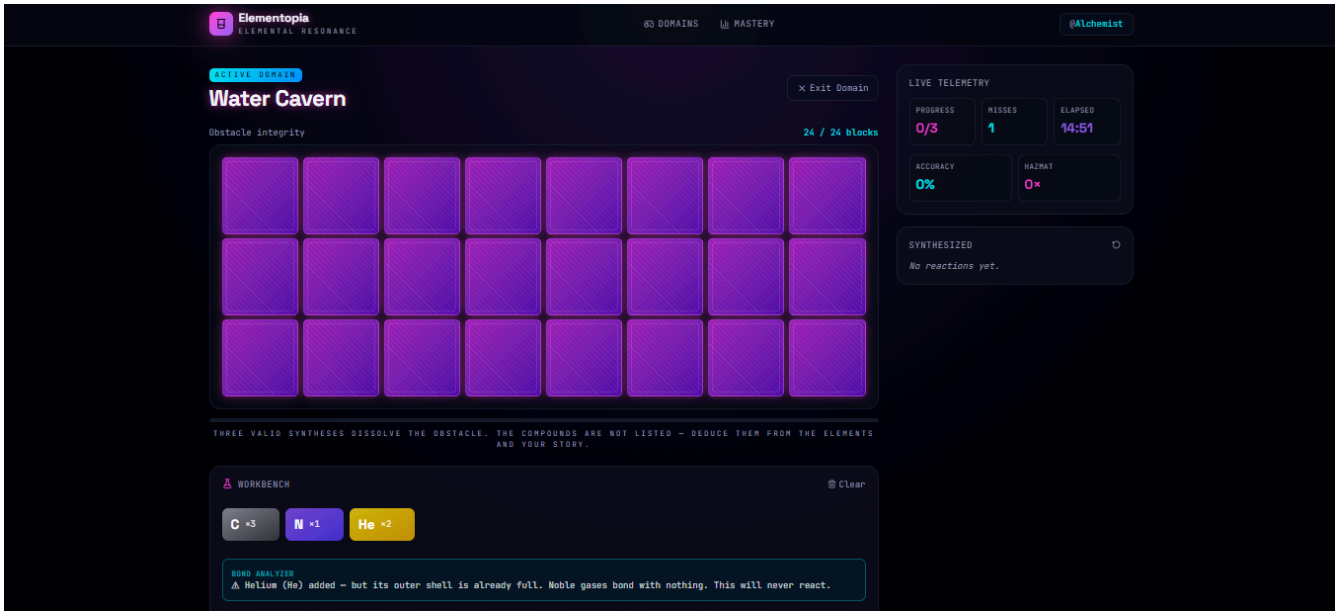


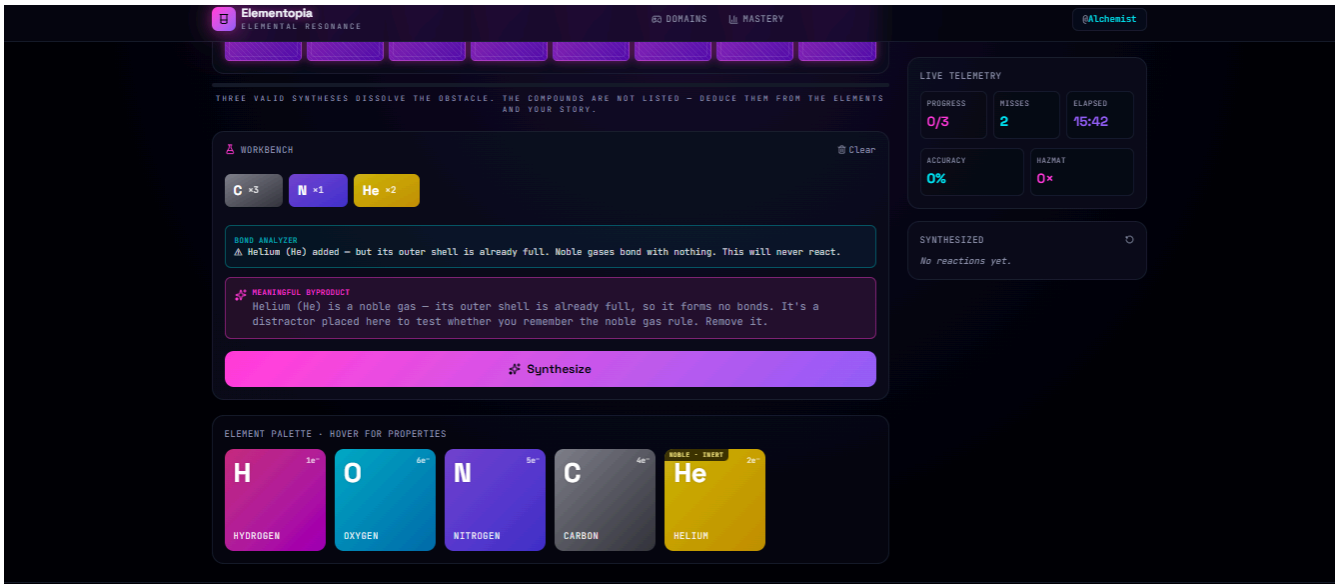
- Data Design
    - ERD or schema:



1.2 Meaningful Byproduct Diagnostic Feedback

- User Interface Design

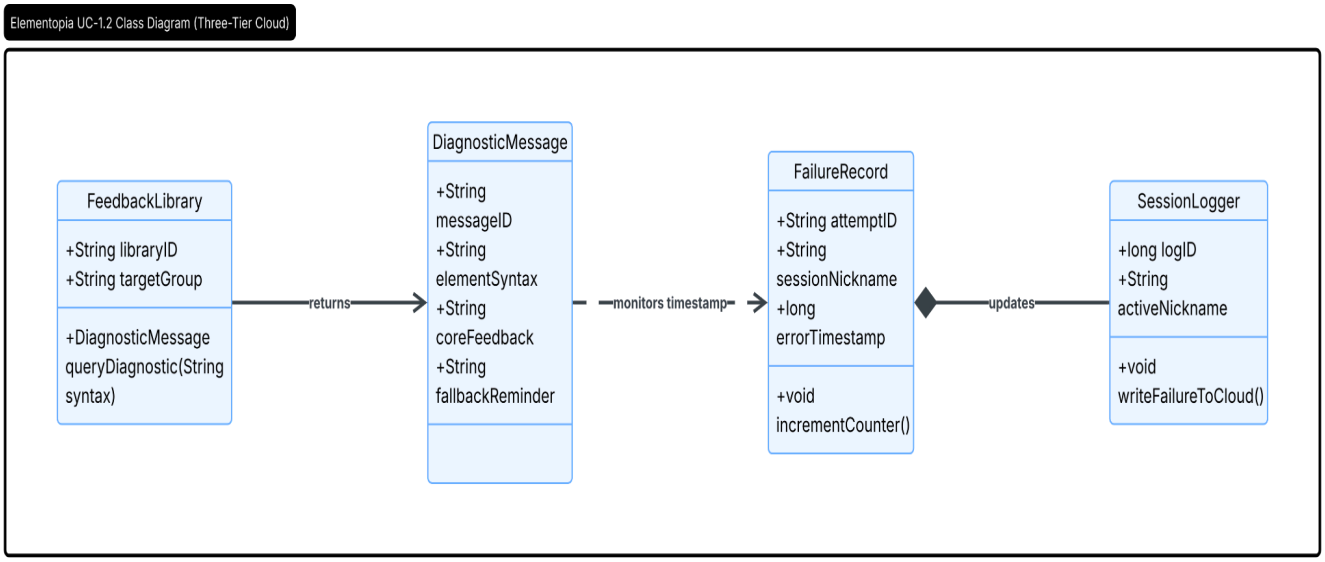




- Front-end component(s)
    - **Component Name:** `DiagnosticAlertOverlay.jsx`
        - **Description and purpose:** This component listens for invalid reaction responses from the backend validation layers. It conditionally overlays the active puzzle arena, rendering the chemistry-based feedback message (e.g., *"Inert Gas detected. No electron exchange occurred."*) directly inside the environment space, while advancing the session's hidden brute-force monitoring counter.
        - **Component type or format:** React.js functional component utilizing context state listeners and conditional canvas animation frames.
    - **Component Name:** `FeedbackLibraryContext.jsx`
        - **Description and purpose:** Provides a centralized frontend data state management context that maps general fallback chemistry principles (such as group or period attributes) if the backend returns a generic error code without a specific predefined reaction match.
        - **Component type or format:** React Context Provider (`.jsx`) wrapping the active learning Domain UI frame.
- Back-end component(s)
    - **Component Name:** DiagnosticController.java
        - **Description and purpose:** Exposes the backend API routing paths tasked with receiving combination inputs and handling error payload deliveries when validation tasks return failures.
        - **Component type or format:** Spring Boot @RestController managing REST endpoints over HTTPS channels.
    - **Component Name:** FeedbackLibraryService.java
        - **Description and purpose:** Evaluates the incorrect combination syntax and queries the system's predefined chemical properties library. If a precise relationship match exists, it retrieves the corresponding micro-lesson; otherwise, it dynamically

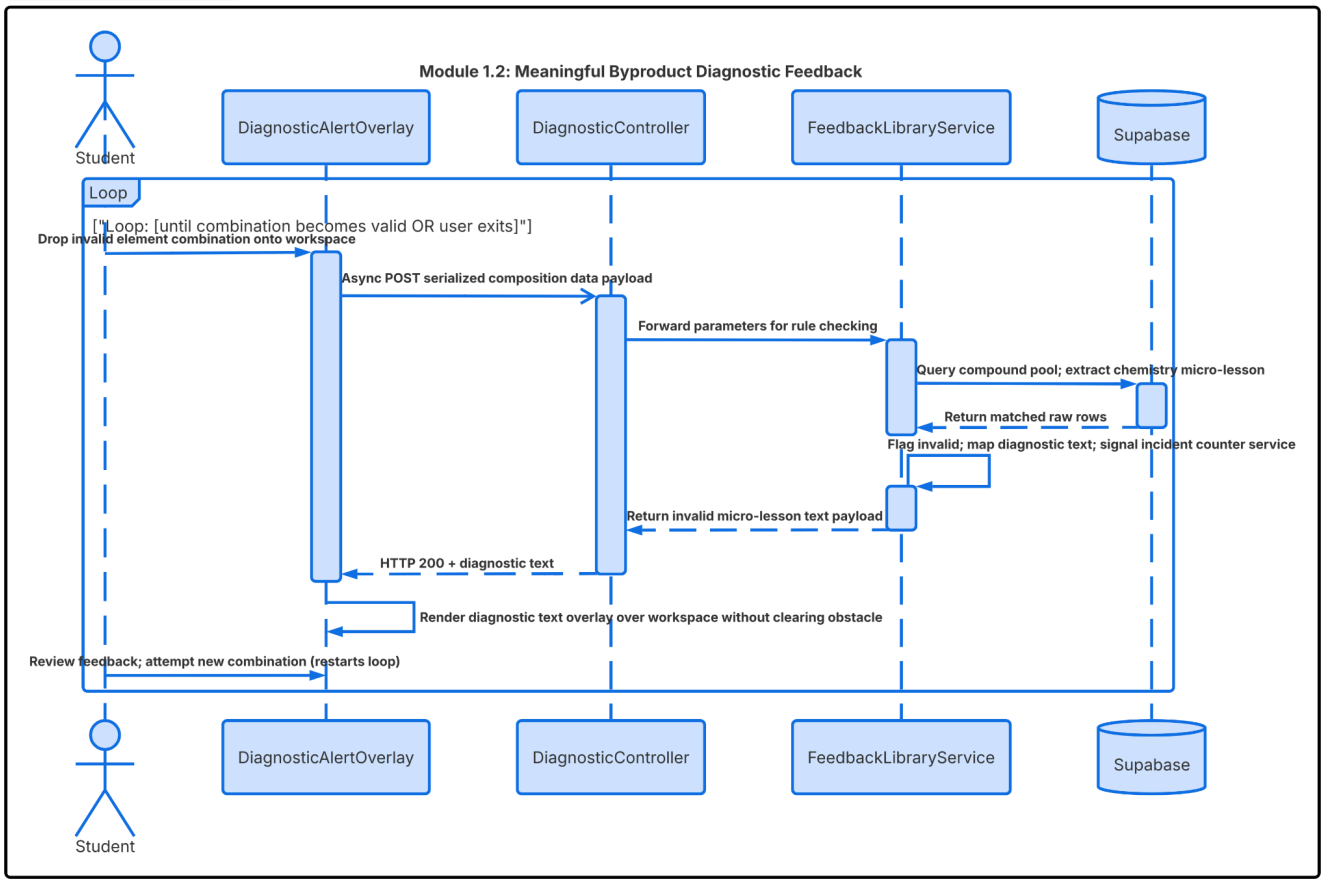
generates a general periodic group or period principle reminder.
    - **Component type or format:** Spring Boot @Service component executing business rule diagnostic logic.
- **Component Name:** IncidentCounterService.java
    - **Description and purpose:** Automatically logs every individual failed attempt and its corresponding millisecond timestamp directly to the session history cache. It increments the operational telemetry counters that feed into the background sliding-window brute-force failsafe monitor.
    - **Component type or format:** Spring Boot @Service handling transactional application state updates.

- Object-Oriented Components
    - Class Diagram:



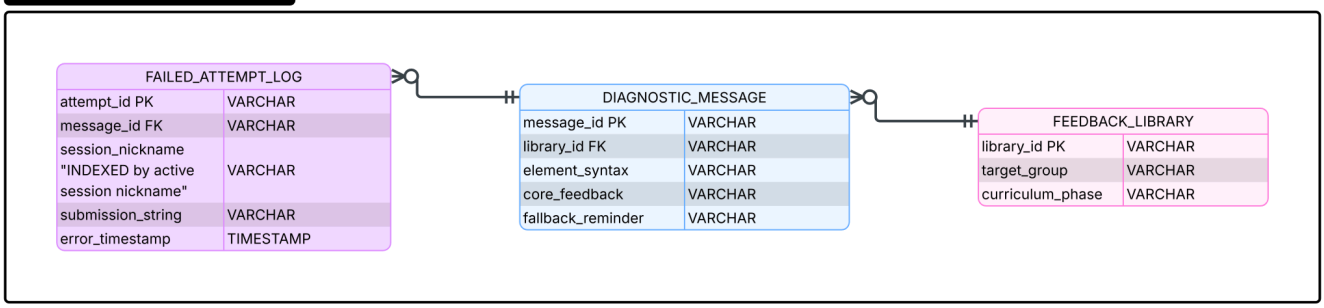
    - Sequence Diagram:

Module 1.2: Diagnostic Feedback Loop



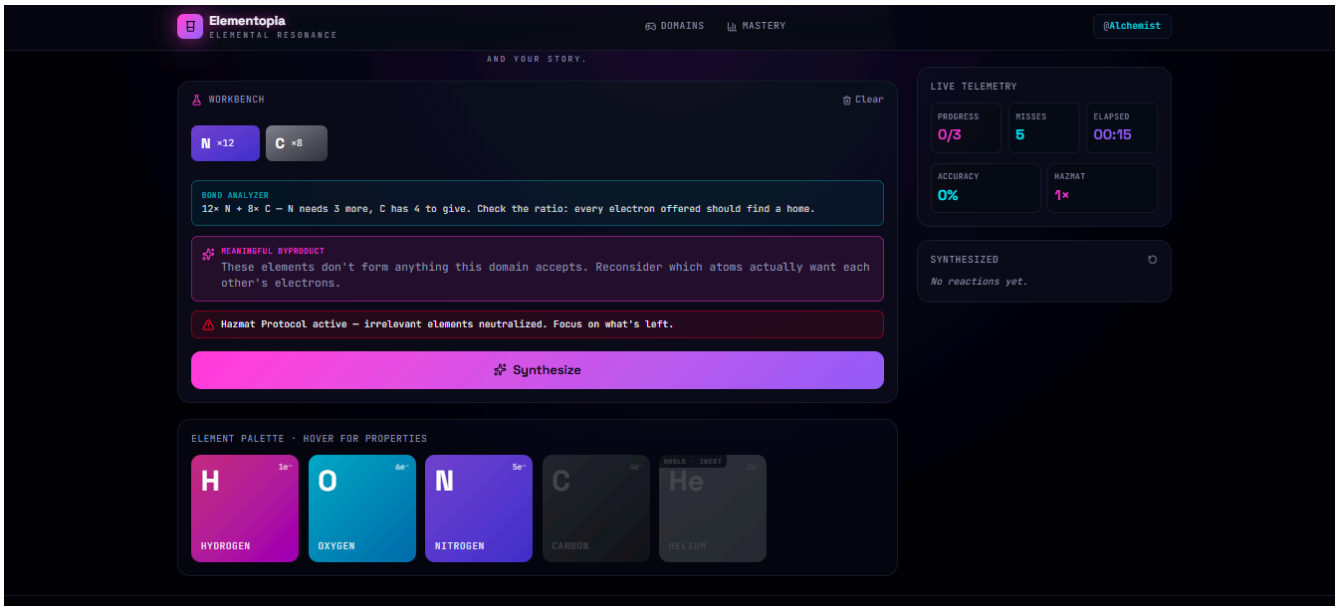
- Data Design
    - ERD or schema:

Elementopia Module 1.2 Diagnostic Error Mapping ERD



1.3 Brute-Force Failsafe

- User Interface Design



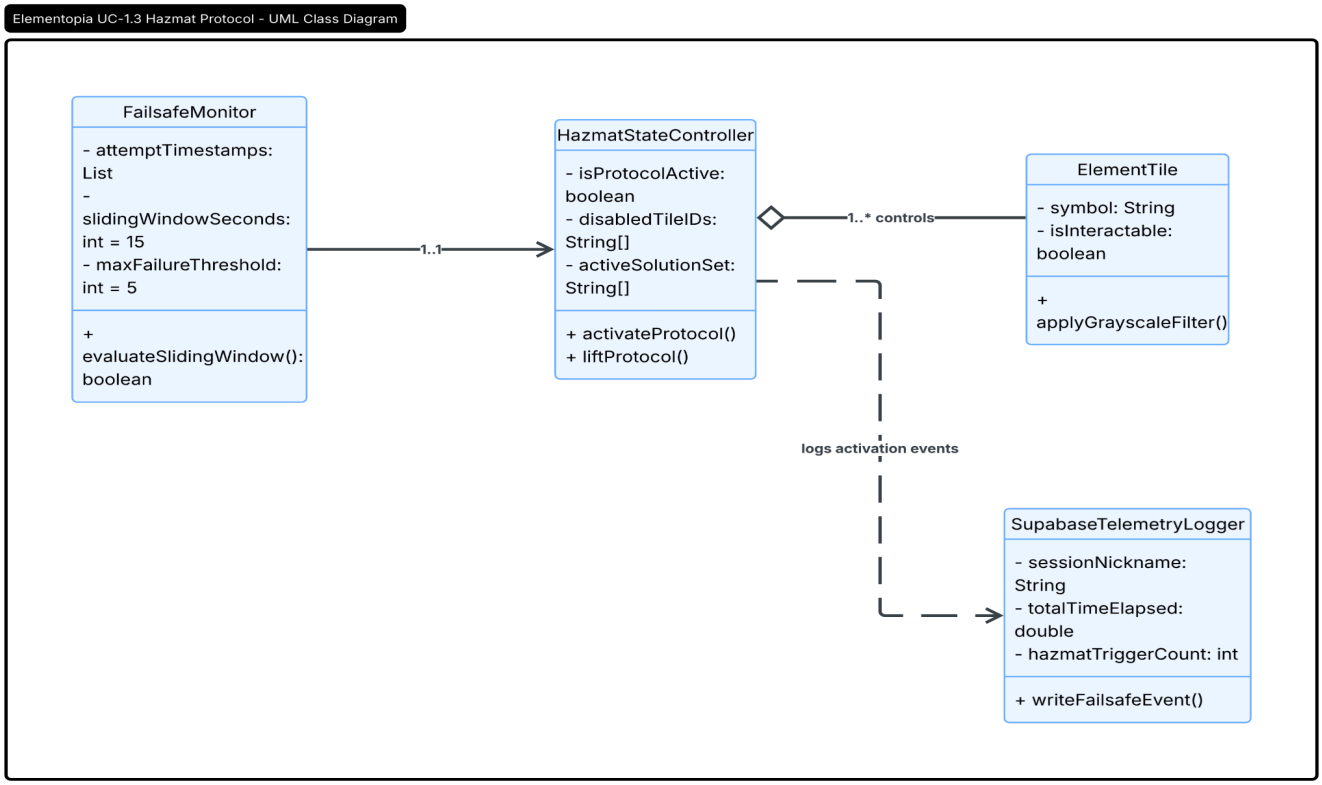
- Front-end component(s)
  - **Component Name:** HazmatOverlayController.jsx
    - **Description and purpose:** This component monitors the local interaction canvas state and intercepts touch/click events when the brute-force conditions are violated. When triggered, it visually grays out and locks all element tiles except those tagged as part of the correct solution path, restricting the student's interaction field to prevent aimless guessing.
    - **Component type or format:** High-order React.js component wrapper applying conditional CSS grayscale filters and state-driven pointer-events locking.
  - **Component Name:** SystemVolatilityBanner.jsx
    - **Description and purpose:** Renders an in-universe alert message text overlay (e.g., *'System Volatility High. Hazardous elements isolated.'*) across the screen immediately upon protocol activation, disappearing only after a valid compound synthesis lifts the intervention.
    - **Component type or format:** Child React.js presentational component triggered by background global state context hooks.
- Back-end component(s)
  - **Component Name:** FailsafeMonitorController.java
    - **Description and purpose:** Provides the API endpoint paths to stream security-focused state changes and notify the system layers of brute-force intervention statuses.
    - **Component type or format:** Spring Boot @RestController handling custom state update payloads.
  - **Component Name:** SlidingWindowEvaluationService.java
    - **Description and purpose:** Runs background verification loops evaluating transaction logs. It analyzes the timestamps of incoming validation requests from

the same user nickname, calculating whether 5 or more failed combinations occurred within a 15-second sliding window. If true, it alters the active session context flag to enable the protocol.

- **Component type or format:** Spring Boot @Service component executing business rules logic equations.

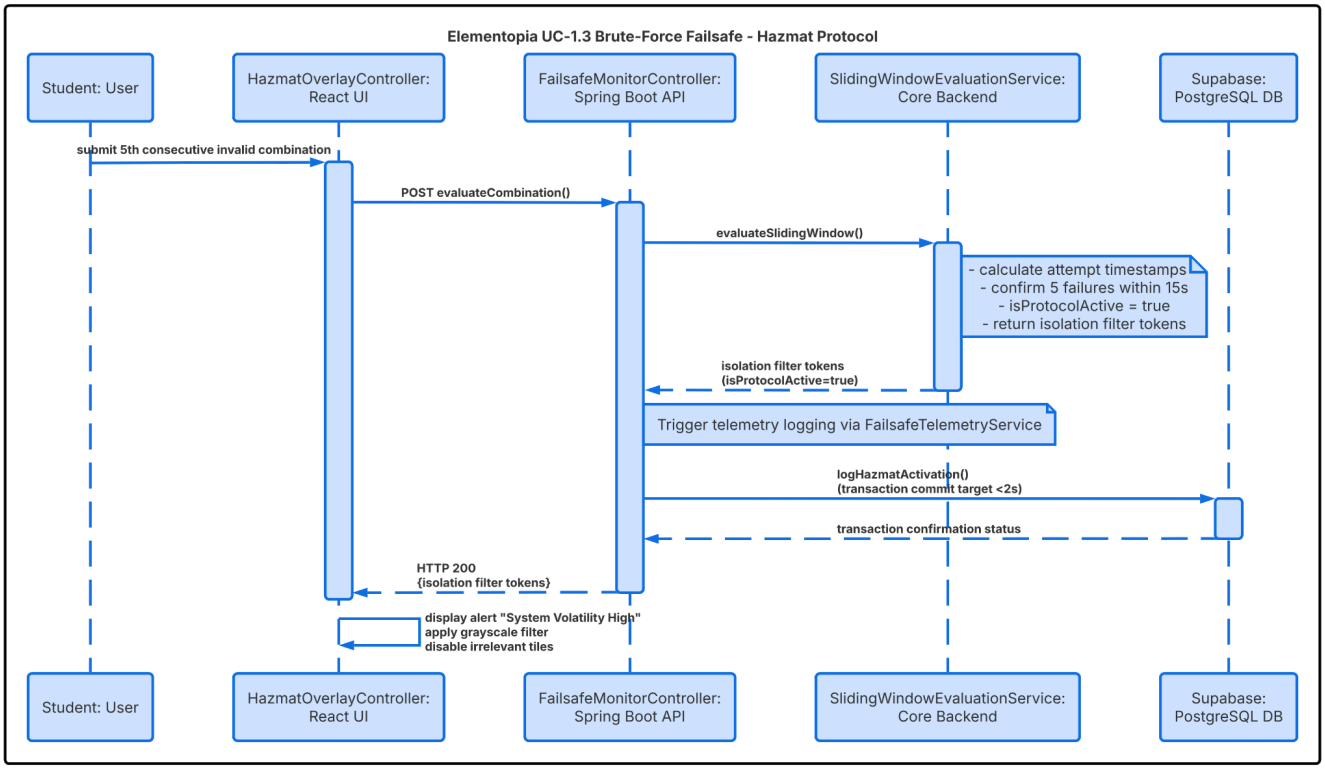
- **Component Name:** FailsafeTelemetryService.java
  - **Description and purpose:** Registers the precise activation and deactivation metrics of the Hazmat Protocol, logging the timestamp parameters safely to cloud storage under 2 seconds.
  - **Component type or format:** Spring Boot @Service controlling transaction mapping data bindings.

- Object-Oriented Components
  - Class Diagram:



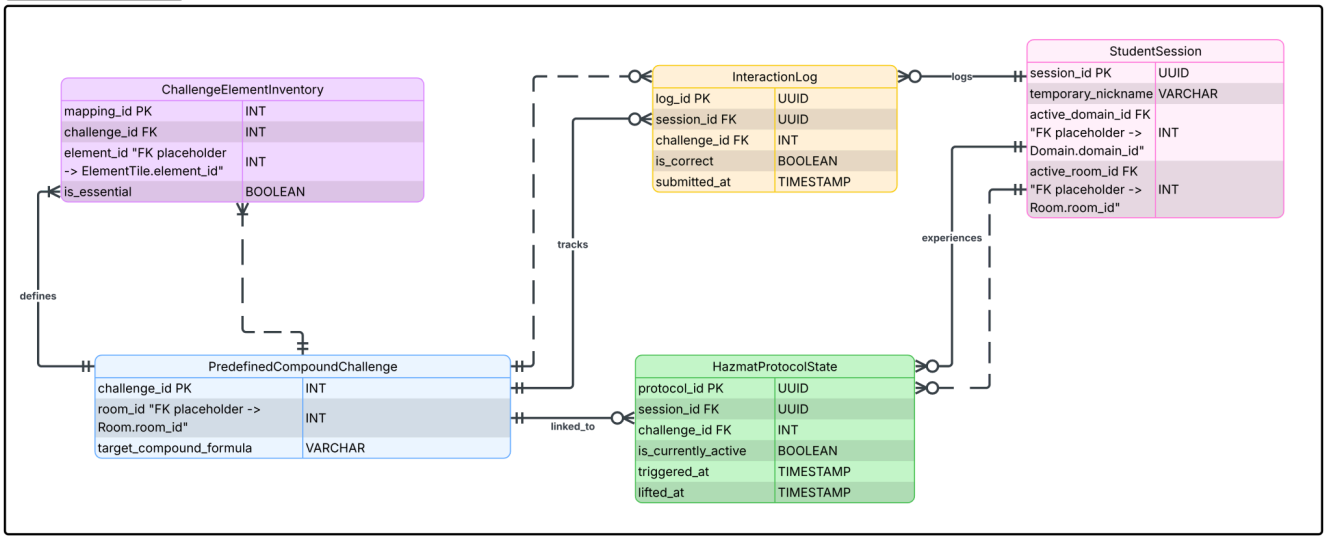
  - Sequence Diagram:

Elementopia UC-1.3 Brute-Force Failsafe - Hazmat Protocol



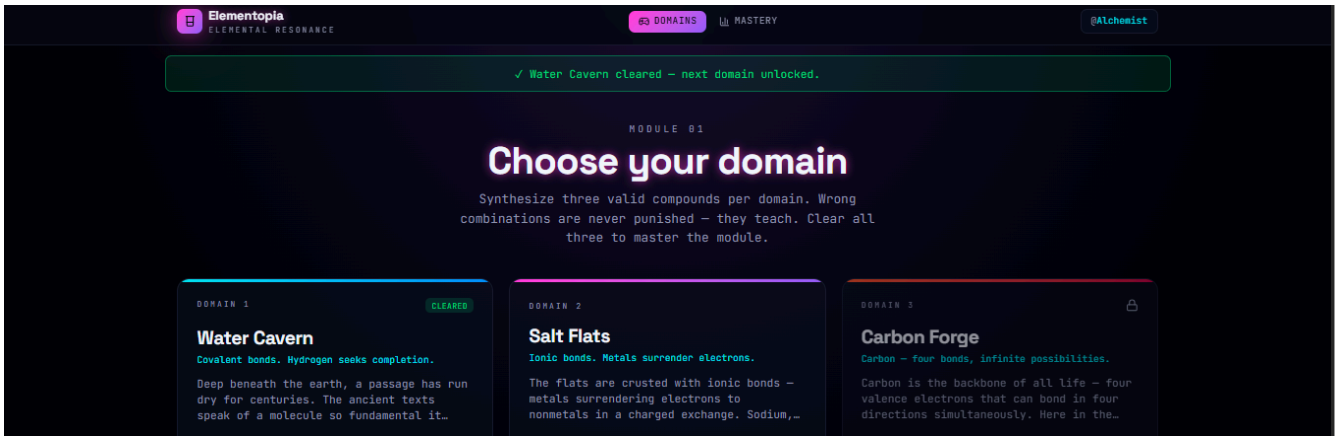
- Data Design
    - ERD or schema:

Elementopia 1.3: Hazmat Protocol ERD



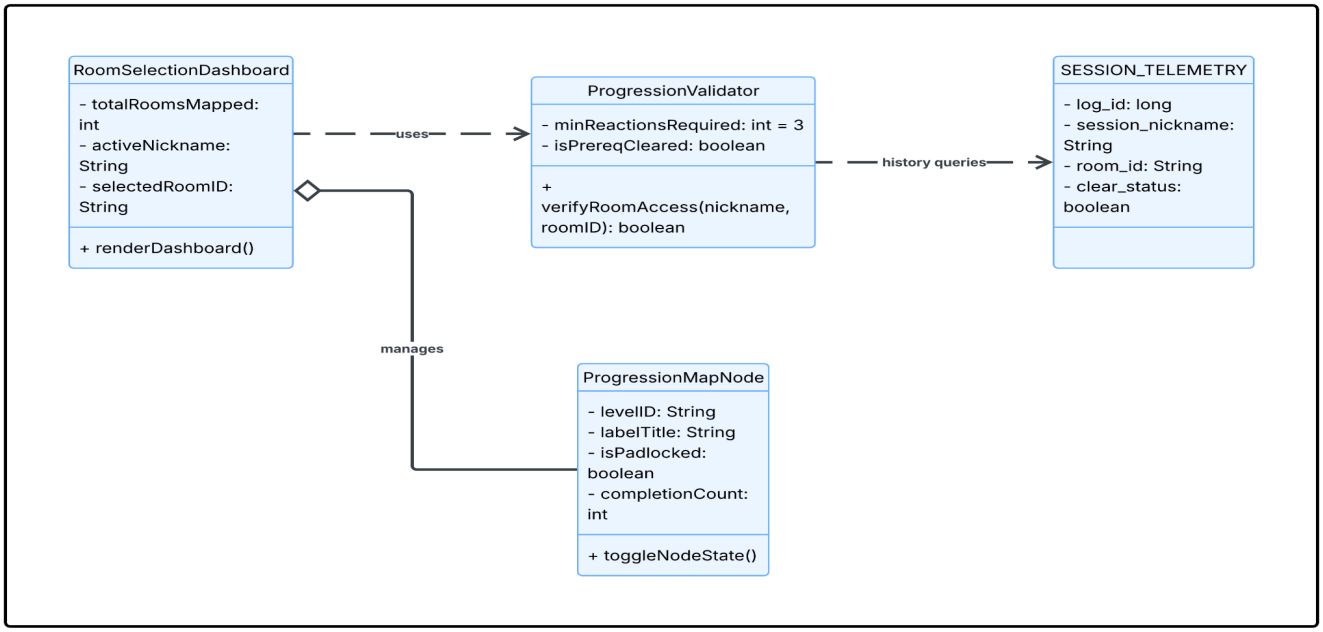
1.4 Learning Room Progression

- User Interface Design



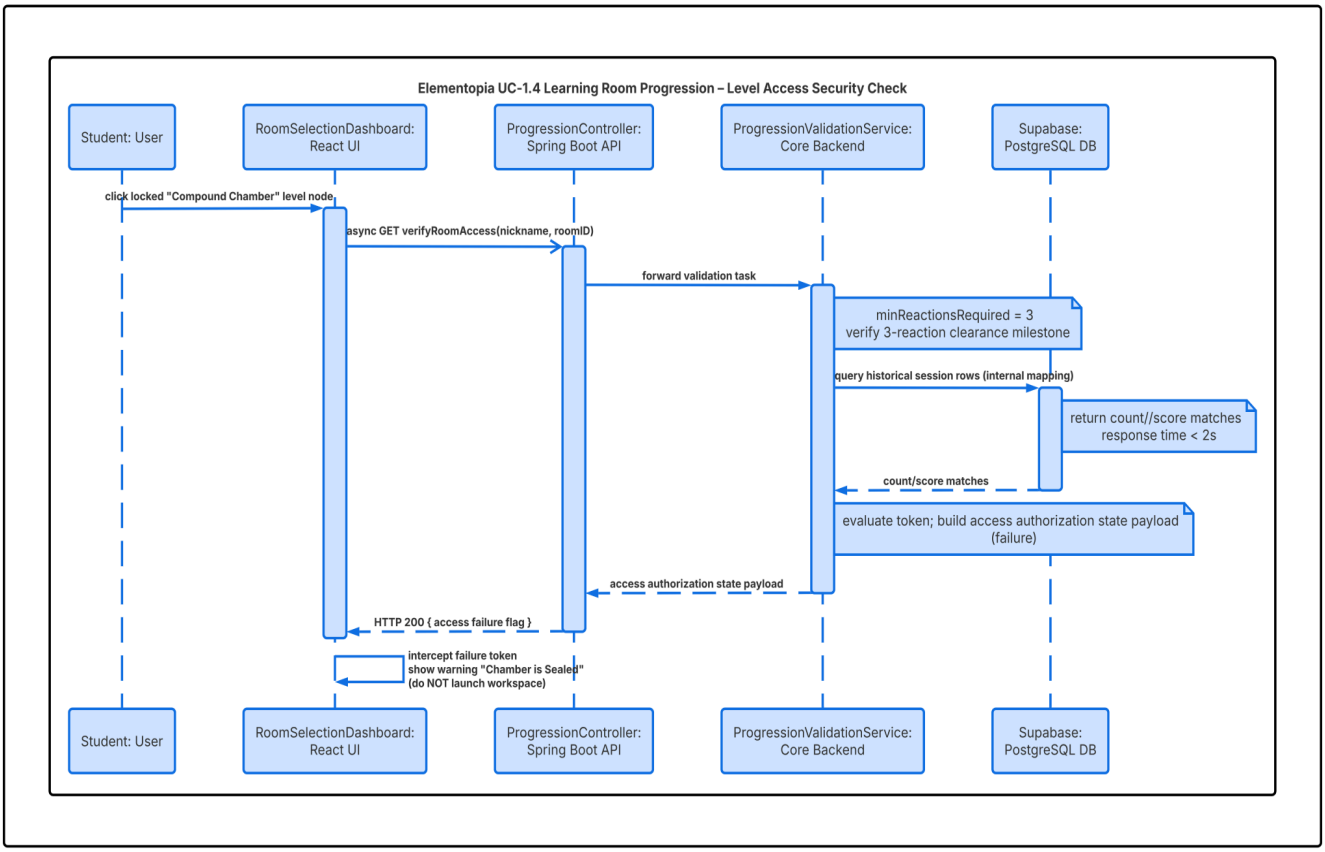
- Front-end component(s)
    - **Component Name:** RoomSelectionDashboard.jsx
        - **Description and purpose:** Renders the map screen layout showing all foundational Element Rooms and advanced Compound Chambers. It queries the user profile state, reads locked or unlocked conditions, and intercepts unauthorized pointer selection attempts on sealed rooms.
        - **Component type or format:** React.js functional view handling screen-width scaling from 360px to 1920px.
    - **Component Name:** ProgressionMapNode.jsx
        - **Description and purpose:** Renders individual level selector nodes displaying completion statuses, structural metadata labels, and visual padlock state badges.
        - **Component type or format:** Child React.js presentational component.
- Back-end component(s)
    - **Component Name:** ProgressionController.java
        - **Description and purpose:** Exposes endpoints to check lock configurations, parse student entry commands, and return initial puzzle session setups.
        - **Component type or format:** Spring Boot @RestController.
    - **Component Name:** ProgressionValidationService.java
        - **Description and purpose:** Verifies if a student meets prerequisite sequence rules (e.g., minimum 3 completed reactions in foundational rooms) before unlocking subsequent levels.
        - **Component type or format:** Spring Boot @Service component executing business rules logic.
    - **Component Name:** ProgressionTelemetryService.java
        - **Description and purpose:** Streams unlock records and updated cleared milestones directly to database instances under 2 seconds.
        - **Component type or format:** Spring Boot @Service controlling transaction mapping data bindings.
- Object-Oriented Components
    - Class Diagram:

Elementopia UC-1.4 Learning Room Progression - UML Class Diagram

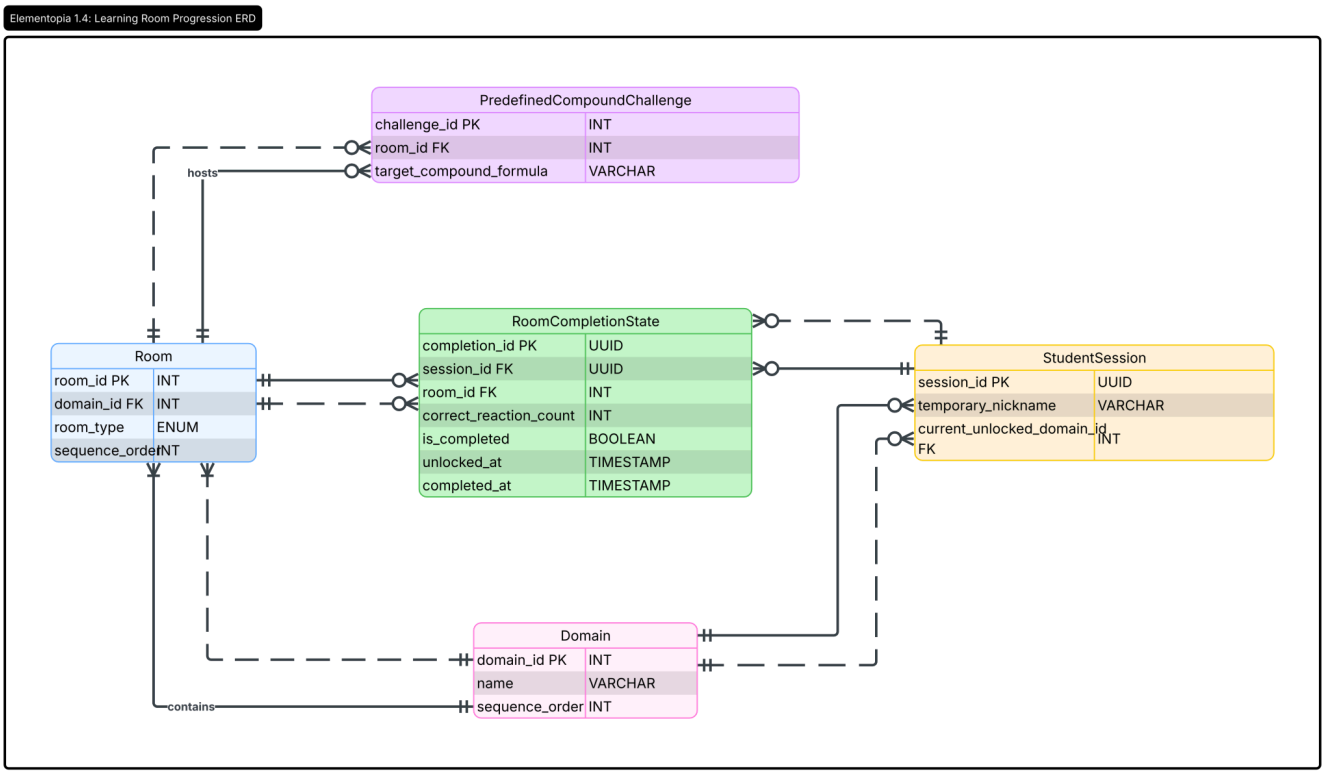


- Sequence Diagram:

Elementopia UC-1.4 Learning Room Progression – Level Access Security Check

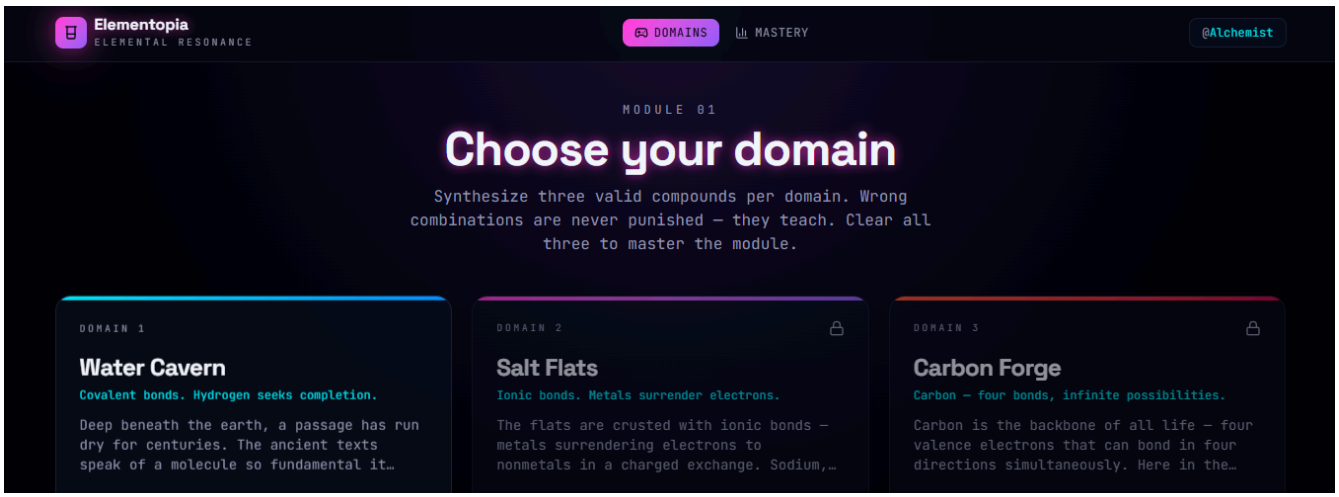


- Data Design
    - ERD or schema:



1.5 Random Compound Generation Per Session

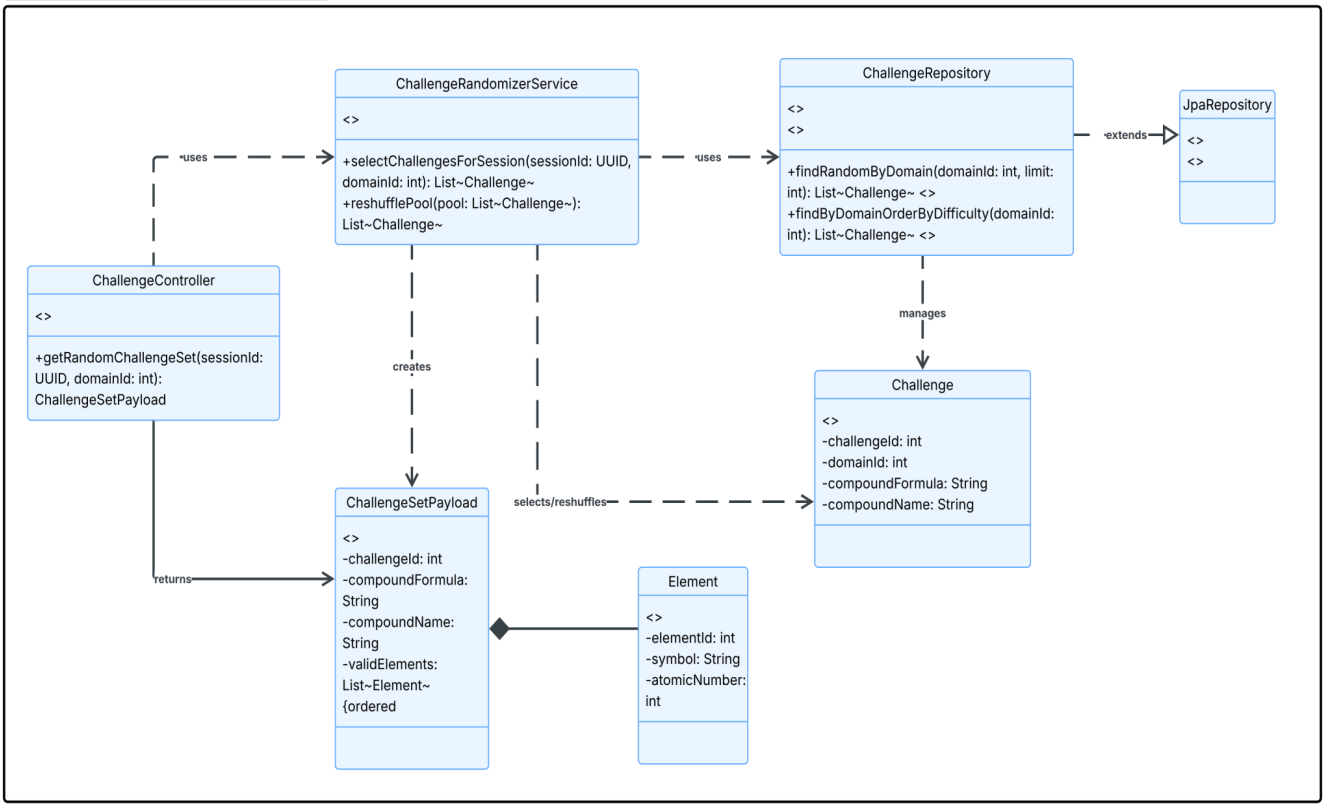
- User Interface Design



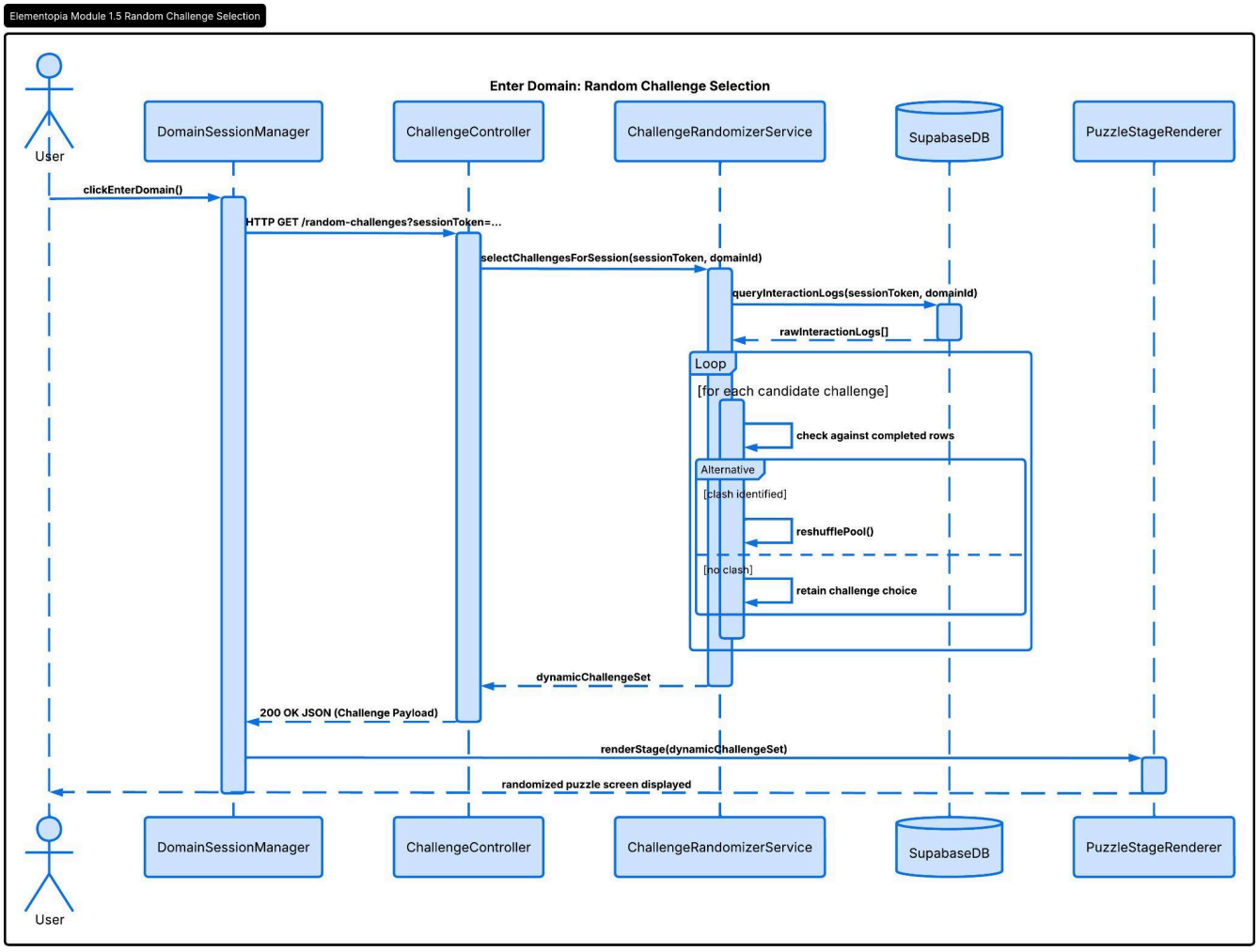
- Front-end component(s)
    - **Component name:** DomainSessionManager
        - **Description and Purpose:** This acts as the container or page-level component that

wraps a self-paced puzzle room session. When a student enters a Domain, this component initializes the session state, requests the randomized, non-repeated challenge set from the backend, and handles client-side state caching for the session's duration.
        - **Component Type or Format:** React Functional Page Component (.jsx / .tsx).
    - **Component name:** PuzzleStageRenderer
        - **Description and Purpose:** Responsible for reading the randomized challenge configuration data passed from the manager and dynamically painting the user interface. It calculates the positions of the dynamic element layouts and updates the background environmental obstacle visuals based on the selected challenge.
        - **Component Type or Format:** React Functional Presentation Component (.jsx / .tsx).
- Back-end component(s)
    - **Component name: ChallengeController**
        - **Description and Purpose:** Exposes the REST API endpoint that the front-end requests when a user clicks into a Domain level. It receives the student's current session identifier and target Domain ID, coordinating with service layers to return the payload.
        - **Component Type or Format:** Spring Boot RestController Java Class.
    - **Component name: ChallengeRandomizerService**
        - **Description and Purpose:** Contains the central algorithmic selection logic. It fetches the standard pool of high school chemistry reactions for the specified domain from the database, checks the user’s history to find previously completed milestones in that current session, runs a reshuffling algorithm to discard duplicates, and selects a fresh, unrepeated challenge set.
        - **Component Type or Format:** Spring Boot @Service Java Class.
- Object-Oriented Components
    - Class Diagram:

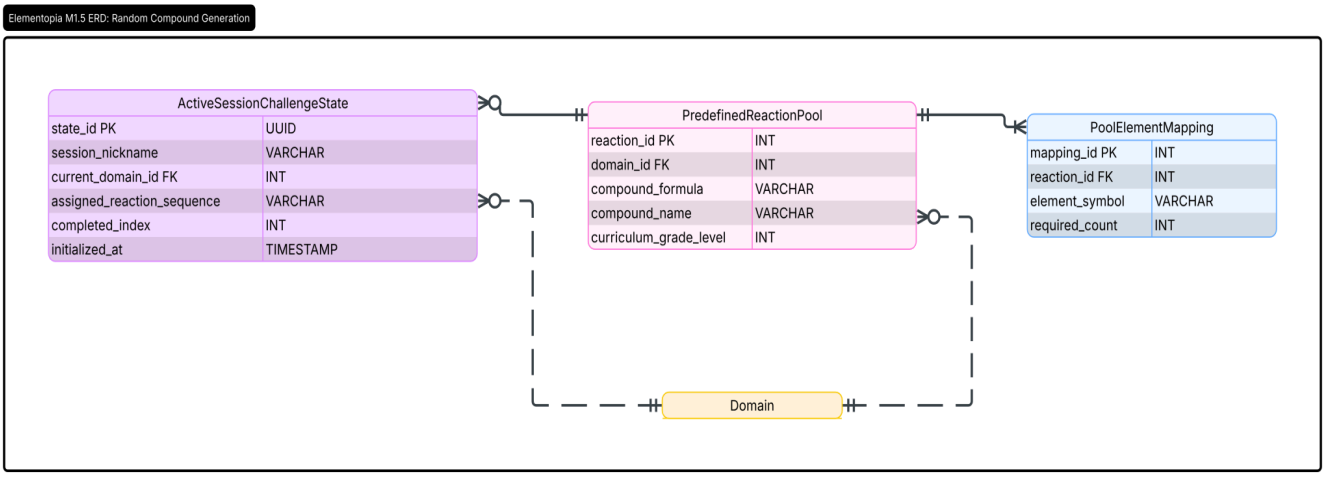
Elementopia 1,5: Randomized Challenge Generation - Class Diagram



- Sequence Diagram



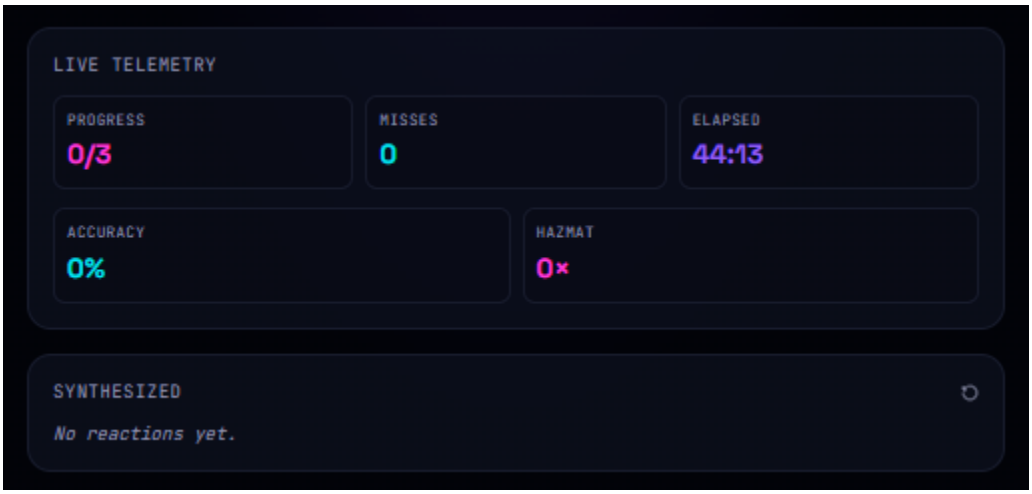
- Data Design
  - ERD or schema



Module 2: Assessment & Progress Module

2.1 Automated Session Performance Logging

- User Interface Design

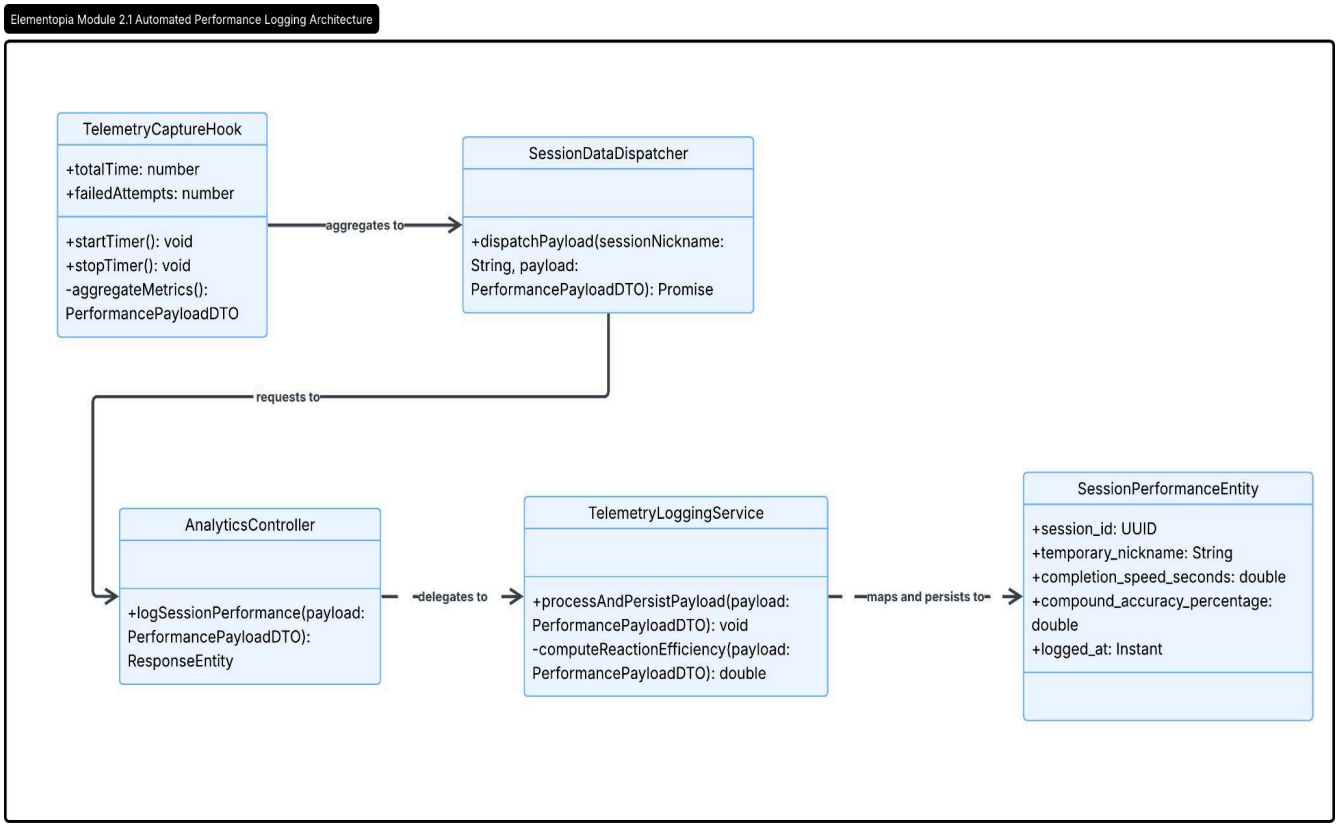


- Front-end component(s)
    - **TelemetryCaptureHook**
        - **Description and Purpose:** A custom hook that monitors active gameplay duration and tracks interaction counters (such as valid syntheses and failed attempts) during a puzzle session. Upon detecting a cleared or completed state, it automatically stops timers, aggregates the behavioral metrics into a performance data transfer object (DTO), and prepares the transactional payload for cloud delivery via the persistence lifecycle layer.
        - **Component Type or Format:** React Custom Hook (.js / .ts).
    - **SessionDataDispatcher**
        - **Description and Purpose:** Manages the network lifecycle for persisting data. It takes the payload generated by the telemetry hook, serializes it, and sends an asynchronous transmission to the backend API. It enforces user-friendly error

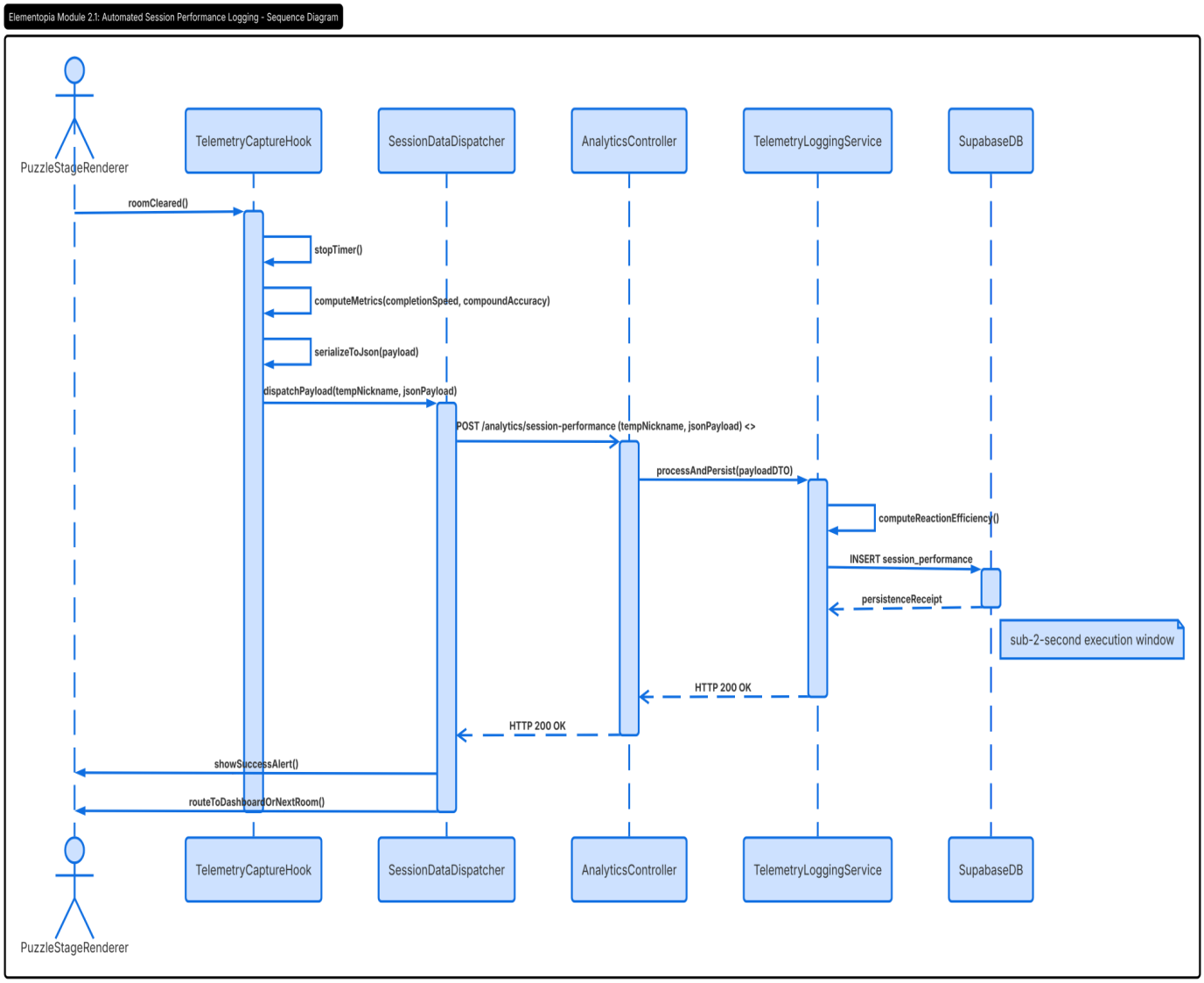
boundaries to prevent application crashes if network anomalies occur, and signals the user interface to transition once a write confirmation is verified.

- **Component Type or Format:** React Service / API Utility Layer Module.

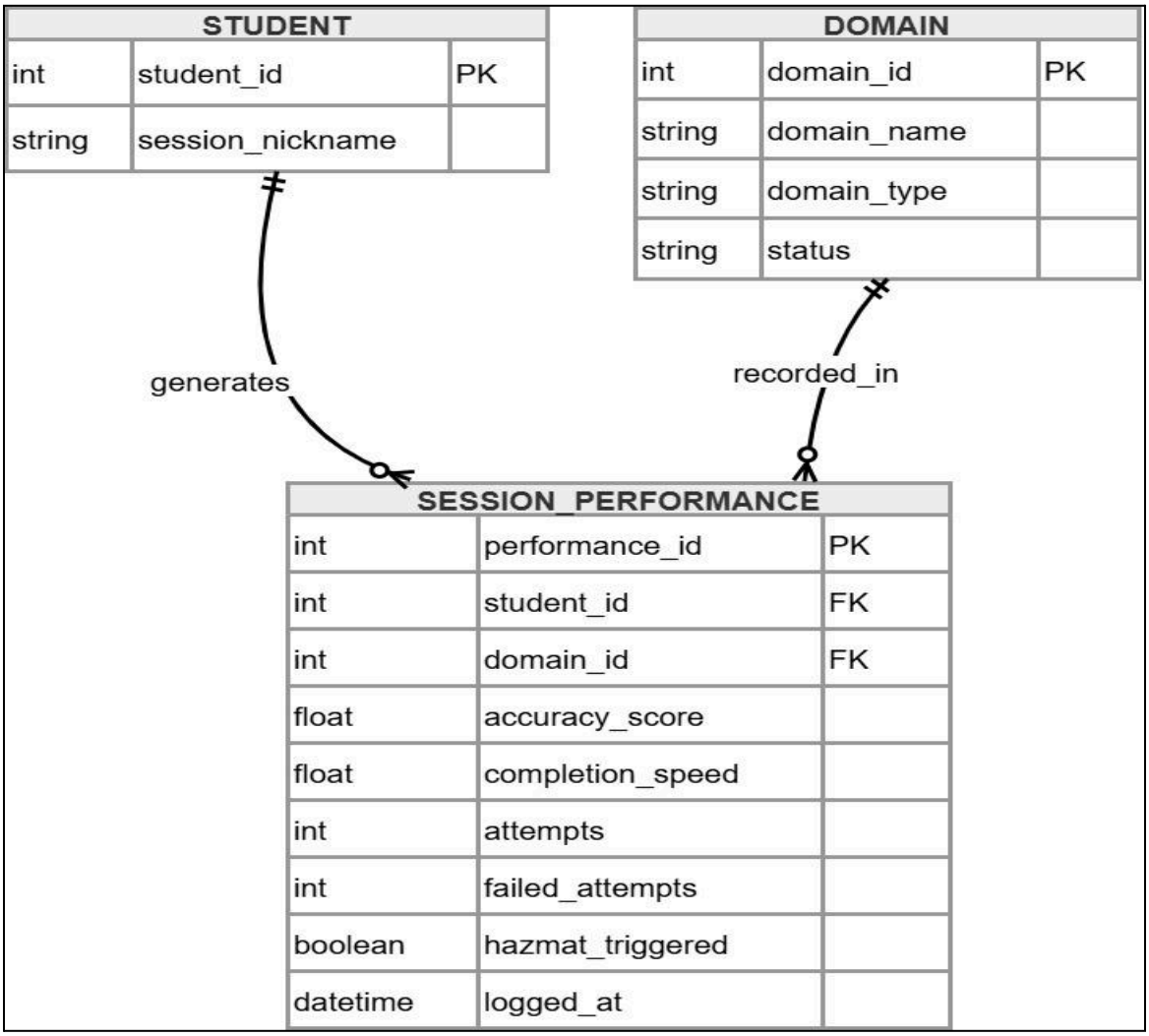
- Back-end component(s)
  - **AnalyticsController**
    - **Description and Purpose:** Exposes secure REST endpoints (e.g., /api/analytics/session/log) to receive raw performance logs from the client application. It maps inbound payloads to structured request data transfer objects and hands them off to the tracking service layer for processing.
    - **Component Type or Format:** Spring Boot RestController Java Class.
  - **TelemetryLoggingService**
    - **Description and Purpose:** Houses the core analytical and database synchronization logic. It receives structural DTOs, performs validation on performance path efficiency computations, associates data records with the zero-registration temporary session nickname, and orchestrates the database write operations to Supabase within the mandated sub-2-second constraint.
    - **Component Type or Format:** Spring Boot @Service Java Class.
- Object-Oriented Components
  - Class Diagram



- Sequence Diagram

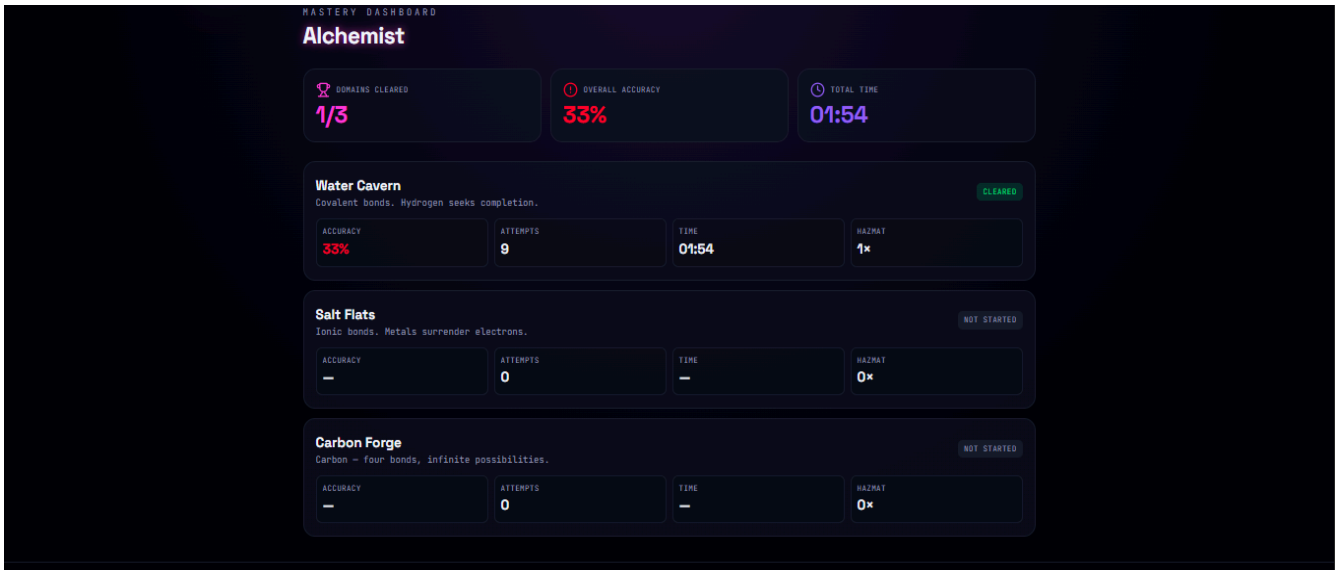


- Data Design
    - ERD or schema



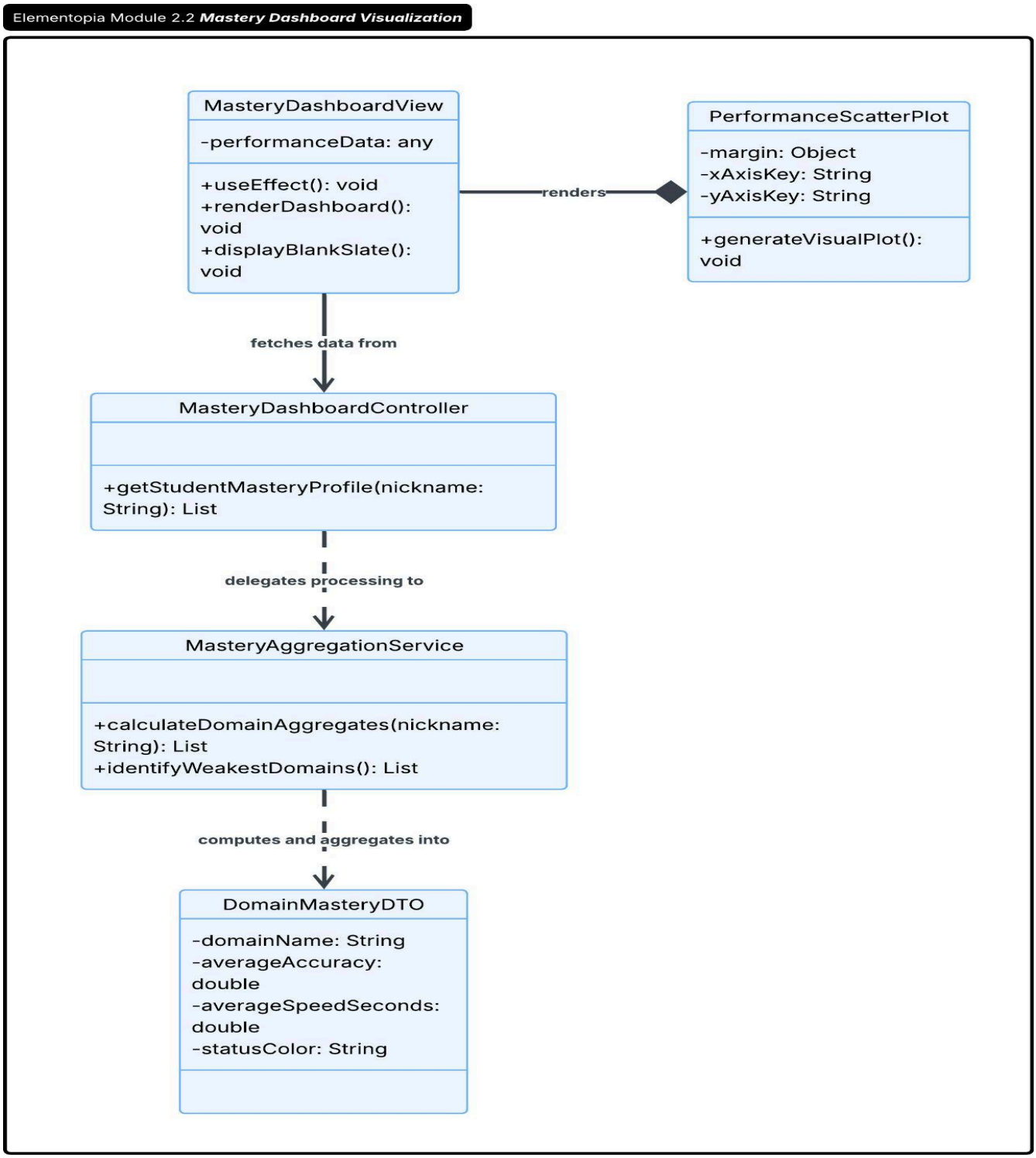
2.2 Mastery Dashboard Visualization

- User Interface Design

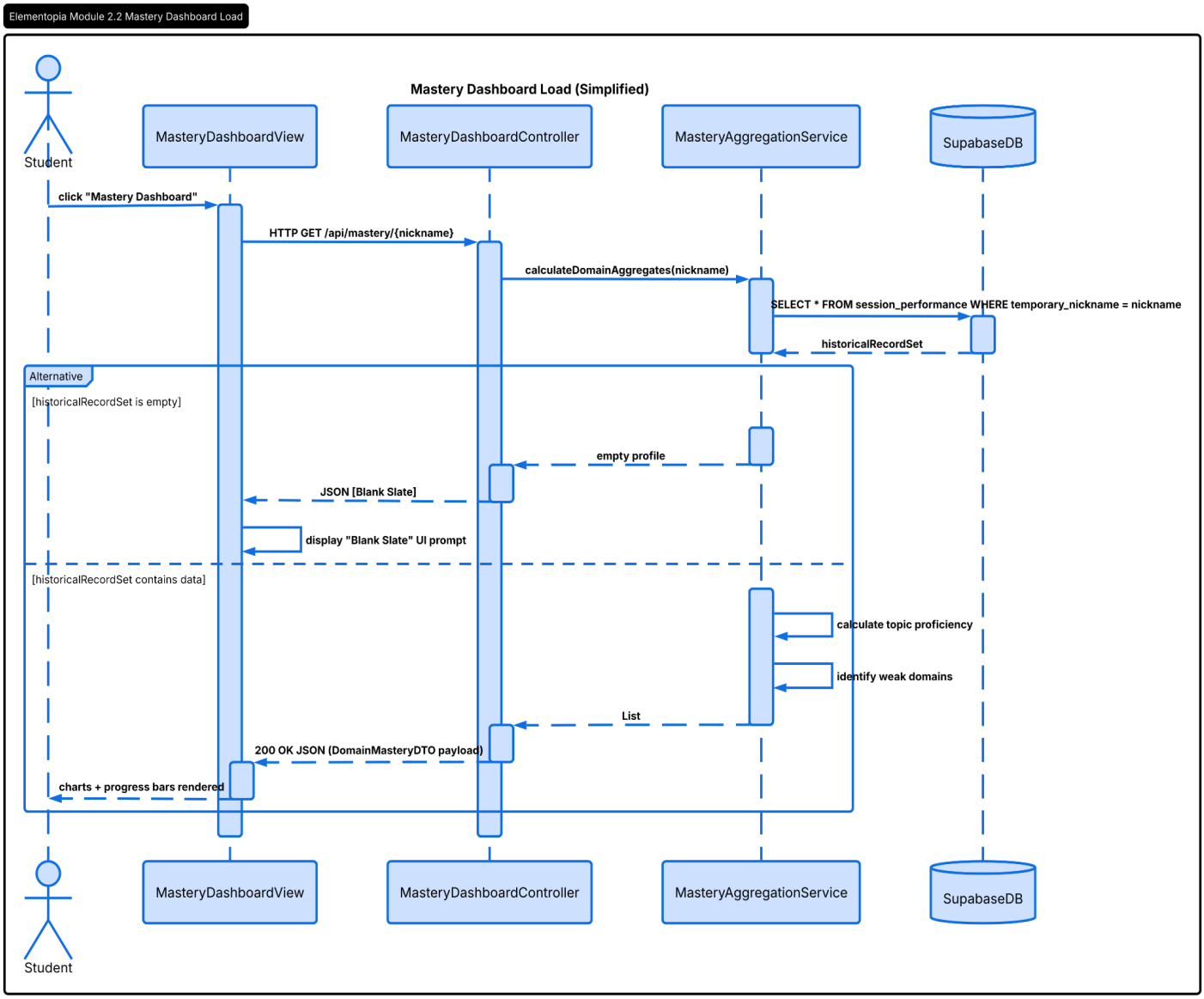


- Front-end component(s)
    - **MasteryDashboardView**
        - **Description and Purpose:** Acts as the primary page container for rendering student performance history. Upon loading, it fetches historical session telemetry from Supabase. If no historical metrics exist, it renders a "Blank Slate" UI prompting the student to join their first level.
        - **Component Type or Format:** React Functional Page Component (.jsx / .tsx).
    - **PerformanceScatterPlot**
        - **Description and Purpose:** A data visualization component that maps historical metrics dynamically. It plots completion speed against compound accuracy per Domain, automatically highlighting weak areas (e.g., rendering low-accuracy sections in red) so students can independently target conceptual gaps.
        - **Component Type or Format:** React Presentation Component utilizing an external charting library (e.g., Recharts or Chart.js).
- Back-end component(s)
    - **MasteryDashboardController**
        - **Description and Purpose:** Exposes the REST API endpoint (e.g., /api/mastery/{nickname}) to retrieve consolidated data profiles for a student based on their temporary session nickname.
        - **Component Type or Format:** Spring Boot RestController Java Class.
    - **MasteryAggregationService**
        - **Description and Purpose:** Handles the core business and analytics logic. It queries raw performance logs from the database, computes aggregate scores for compound accuracy and average completion velocity per Domain, and returns a tailored structural metrics payload optimized for frontend rendering.
        - **Component Type or Format:** Spring Boot @Service Java Class.
- Object-Oriented Components

- Class Diagram

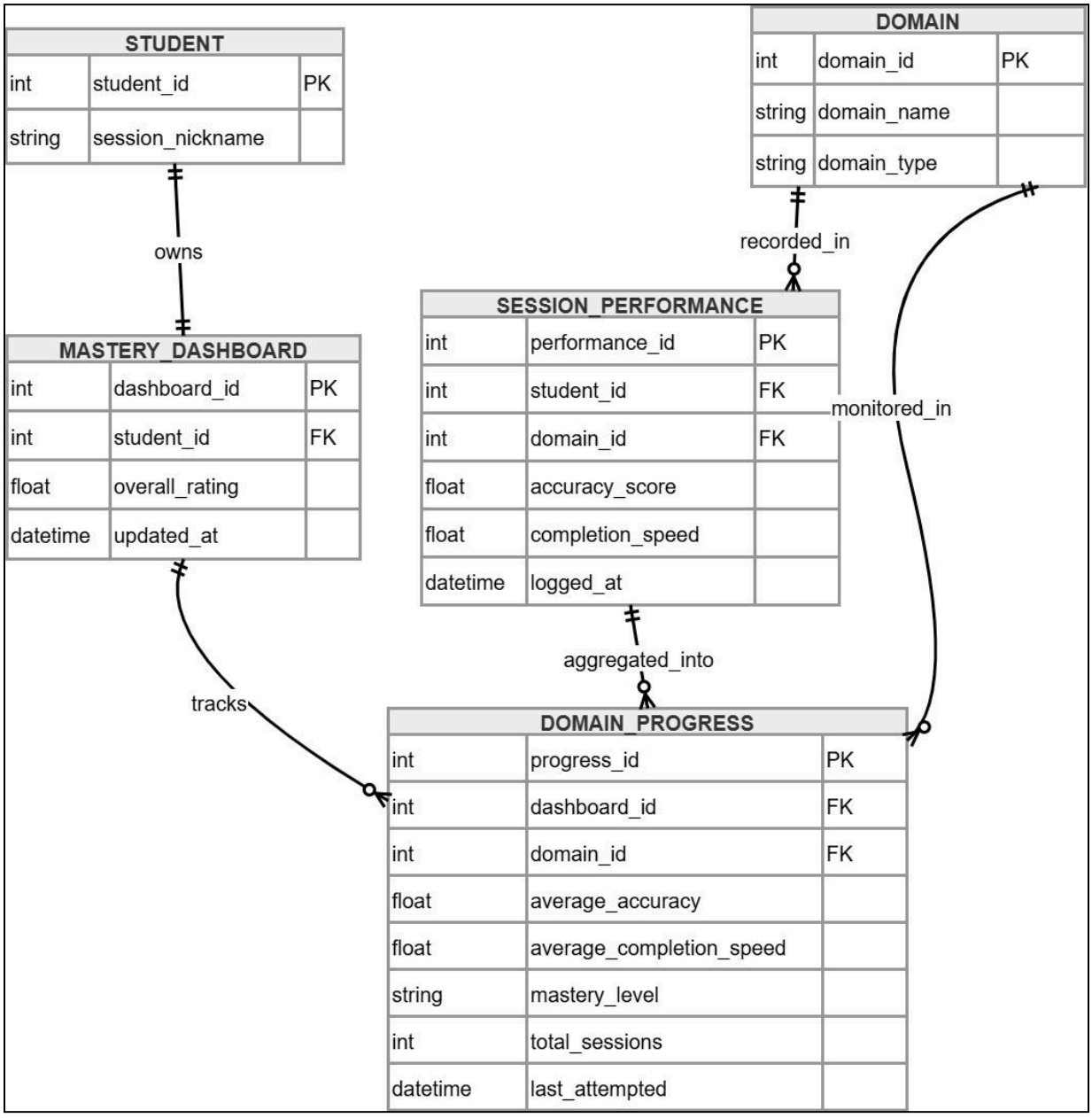


- Sequence Diagram


Elementopia Module 2.2 Mastery Dashboard Load
Mastery Dashboard Load (Simplified)
Student
MasteryDashboardView
MasteryDashboardController
MasteryAggregationService
SupabaseDB
click "Mastery Dashboard"
HTTP GET /api/mastery/{nickname}
calculateDomainAggregates(nickname)
SELECT * FROM session_performance WHERE temporary_nickname = nickname
historicalRecordSet
Alternative
[historicalRecordSet is empty]
empty profile
JSON [Blank Slate]
display "Blank Slate" UI prompt
[historicalRecordSet contains data]
calculate topic proficiency
identify weak domains
List
200 OK JSON (DomainMasteryDTO payload)
charts + progress bars rendered
MasteryDashboardView
MasteryDashboardController
MasteryAggregationService
SupabaseDB
Student

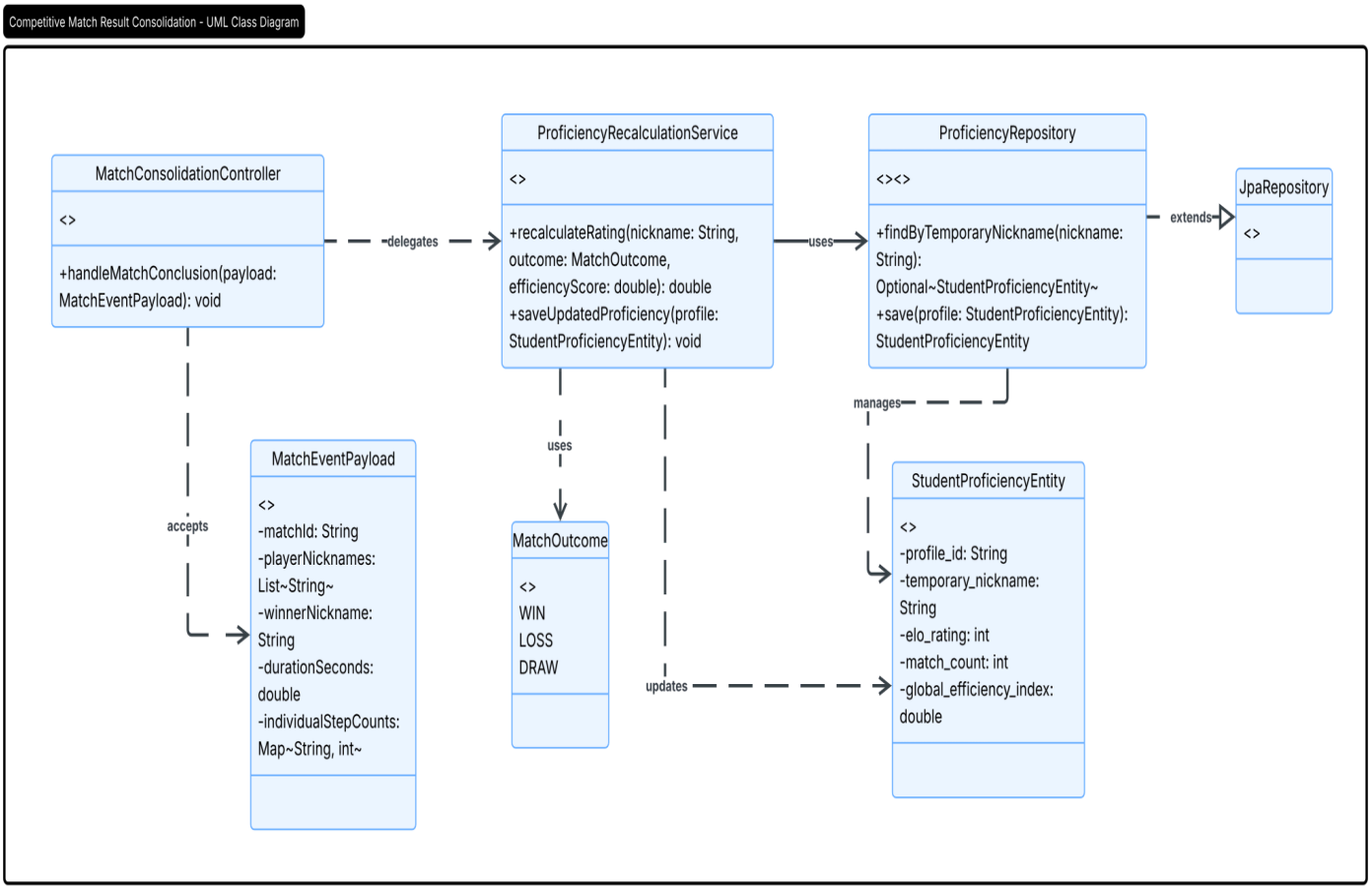
- Data Design

- ERD or schema

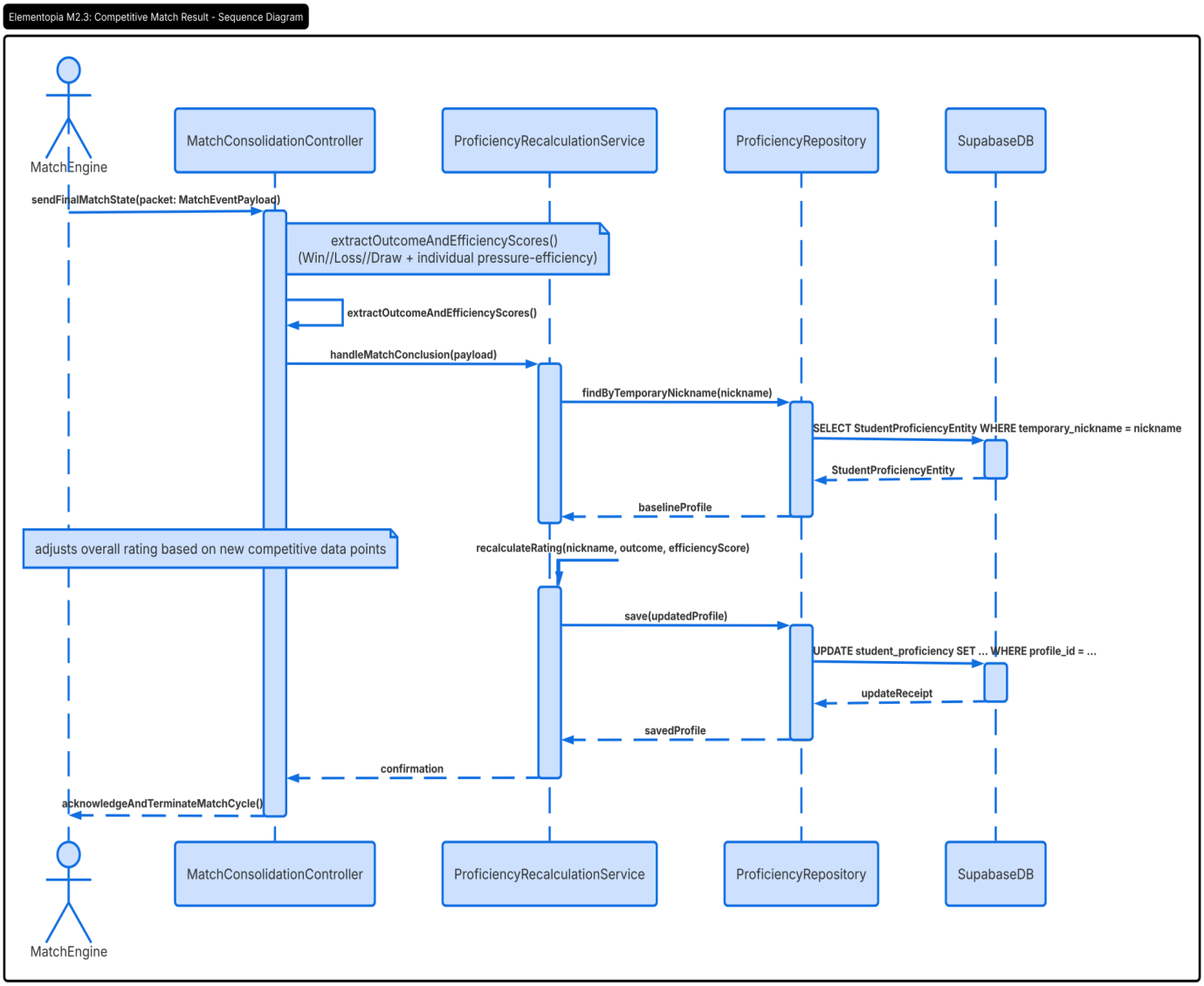


2.3 Competitive Match Result Consolidation

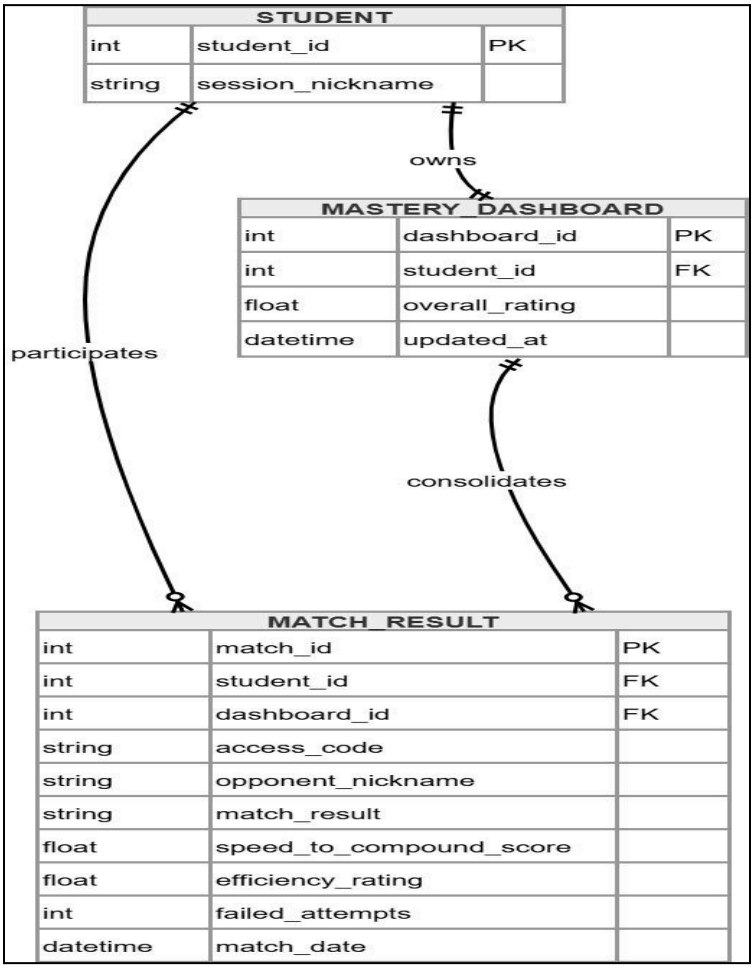
- Back-end component(s)
    - **MatchConsolidationController**
        - **Description and Purpose:** Exposes internal endpoints or acts as the listener boundary that handles incoming match conclusion events from the multiplayer game loop. It extracts the raw match final state (Win/Loss/Draw) along with the player's pressure-efficiency scores and directs them to the consolidation workflow.
        - **Component Type or Format:** Spring Boot RestController / Event Listener Java Class.
    - **ProficiencyRecalculationService**
        - **Description and Purpose:** Houses the core algorithmic rating formulas. It retrieves the student's existing proficiency profile from Supabase, processes the new match results alongside their Speed-to-Compound efficiency metrics, computes the adjusted rating, and pushes the updated data record back to persistent storage.
        - **Component Type or Format:** Spring Boot @Service Java Class.
- Object-Oriented Components
    - Class Diagram



- Sequence Diagram



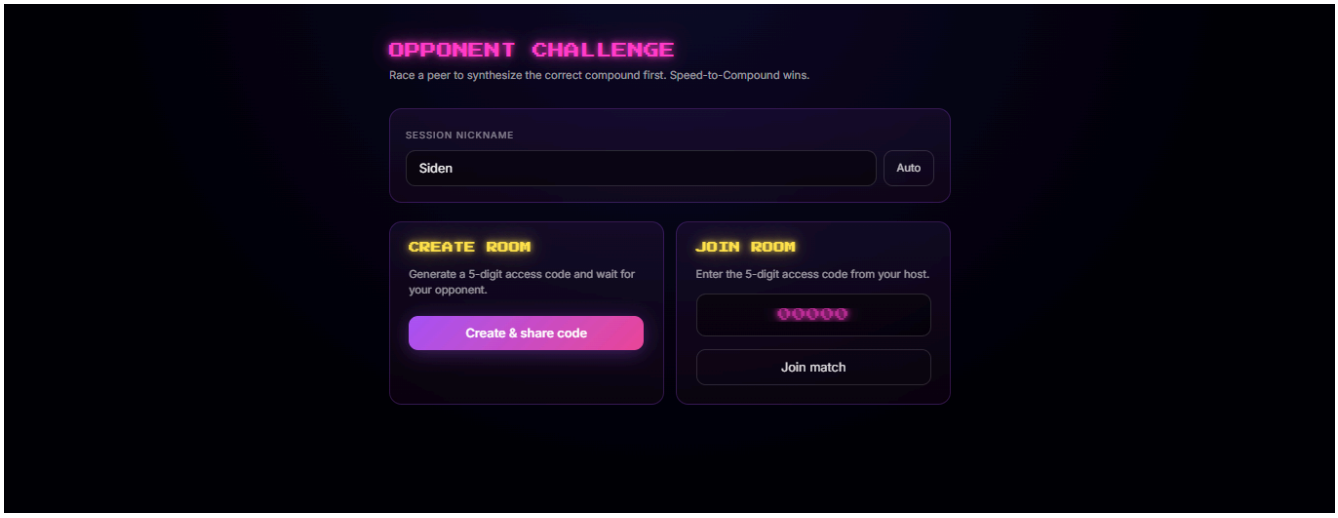
- Data Design
    - ERD or schema



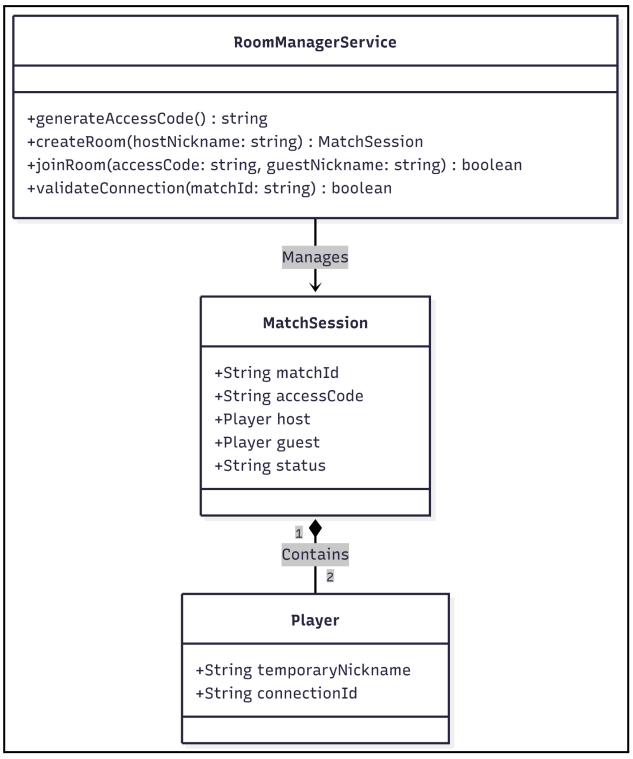
Module 3: Opponent-Based Challenge Module

3.1 Room Creation and Matchmaking

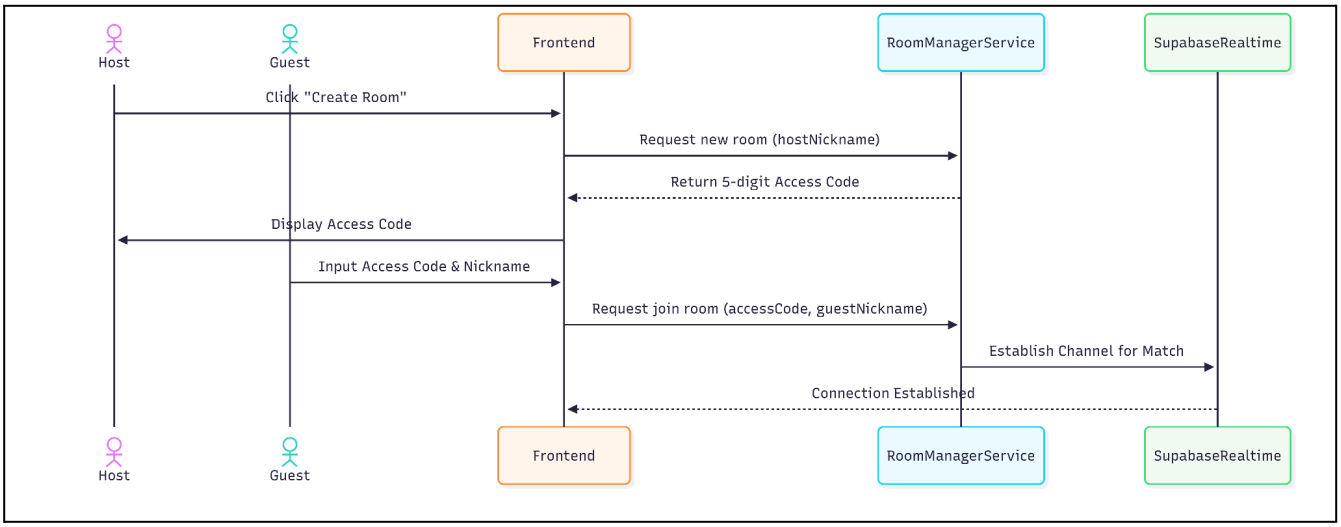
- User Interface Design



- Front-end component(s)
    - MatchmakingController
        - Manages UI state for room creation and joining, displaying auto-generated codes or input forms.
        - React Functional Component
    - CodeInputModal
        - Captures the 5-digit access code and the guest's temporary session nickname.
        - React Modal Component
- Back-end component(s)
    - RoomManagerService
        - Handles generation of the 5-digit code, registers the room session in memory, and establishes WebSocket connections via Supabase Realtime.
        - Spring Boot Service Class
- Object-Oriented Components
    - Class Diagram



- Sequence Diagram

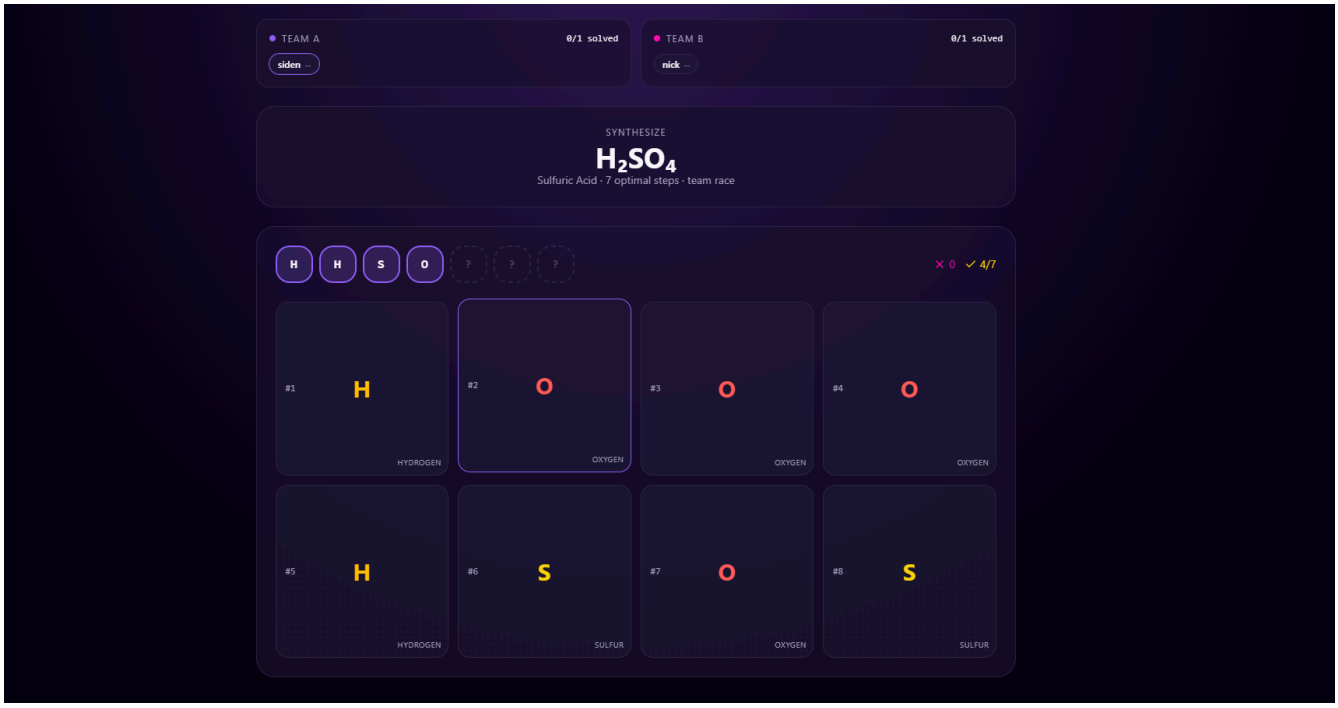


- Data Design
  - ERD or schema

ACTIVE_MATCH_LOBBY			
string	matchId	PK	Unique UUID for the match lobby
string	hostNickname		Session name of creator
string	guestNickname		Session name of joiner
string	accessCode		Auto-generated 5-digit join code
string	status		WAITING, IN_PROGRESS, COMPLETED
timestamp	createdAt		Time the room was created

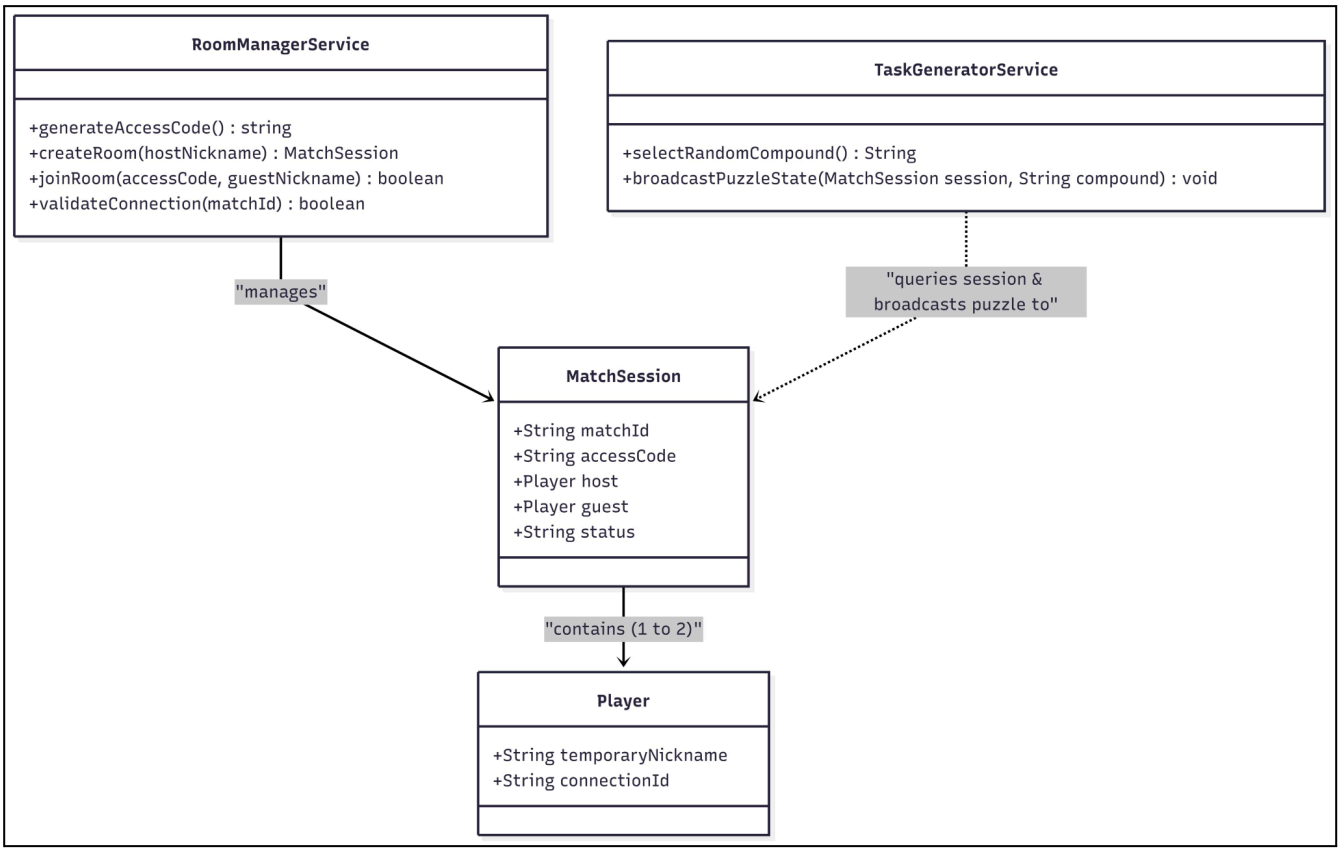
3.2 Synchronized Competitive Task Generation

- User Interface Design

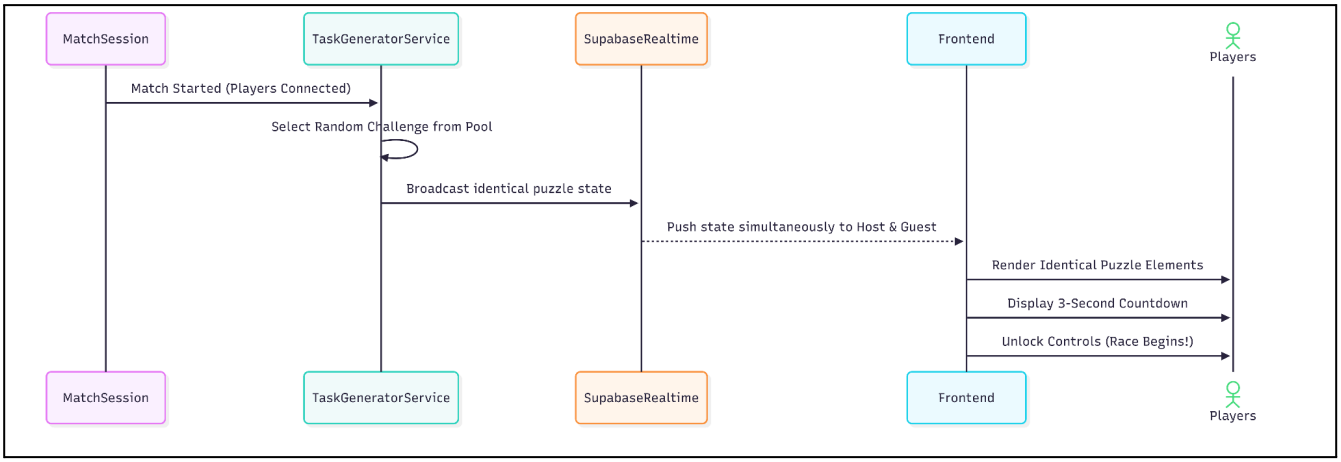


- Front-end component(s)
    - CompetitivePuzzleBoard

        - Renders the environmental obstacle and element inventory. Locks controls during the initial countdown to ensure a fair start.
        - React Functional Component
    - CountdownOverlay
        - Displays the synchronized 3-second visual countdown before unlocking player controls
        - React Component / Animation hook
- Back-end component(s)
    - TaskGeneratorService
        - Selects a randomized compound challenge from the predefined pool and broadcasts it to both clients to ensure 100% identical scenarios.
        - Spring Boot Service Class
- Object-Oriented Components
    - Class Diagram



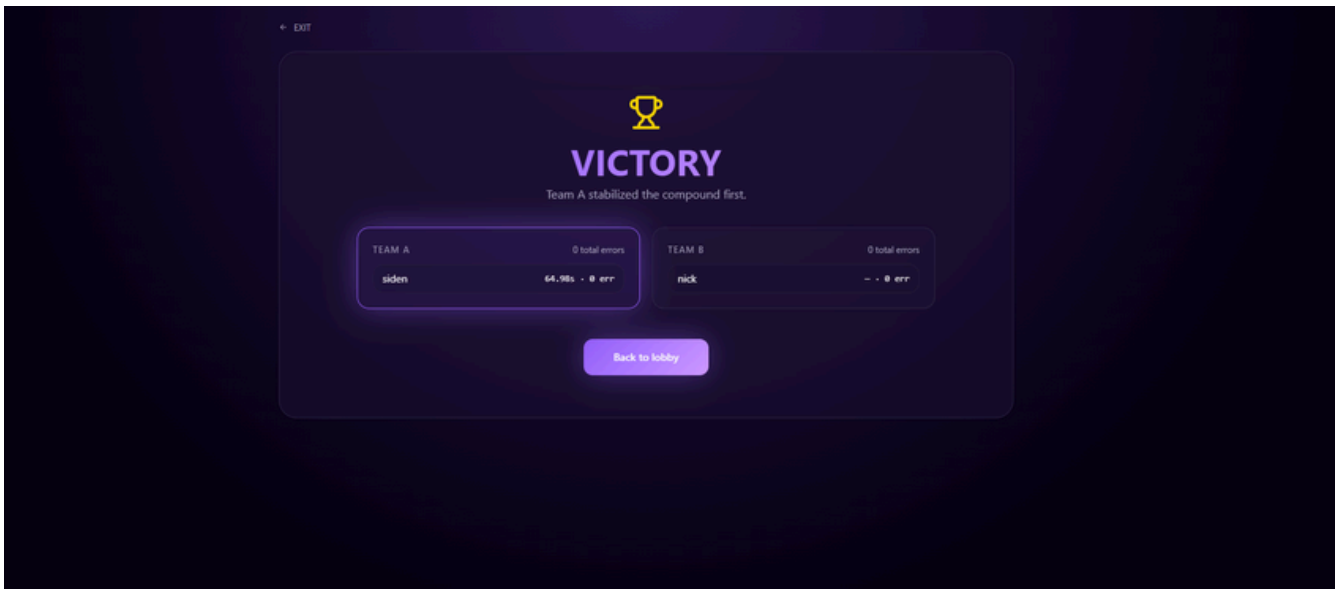
    - Sequence Diagram



- Data Design
    - ERD or schema

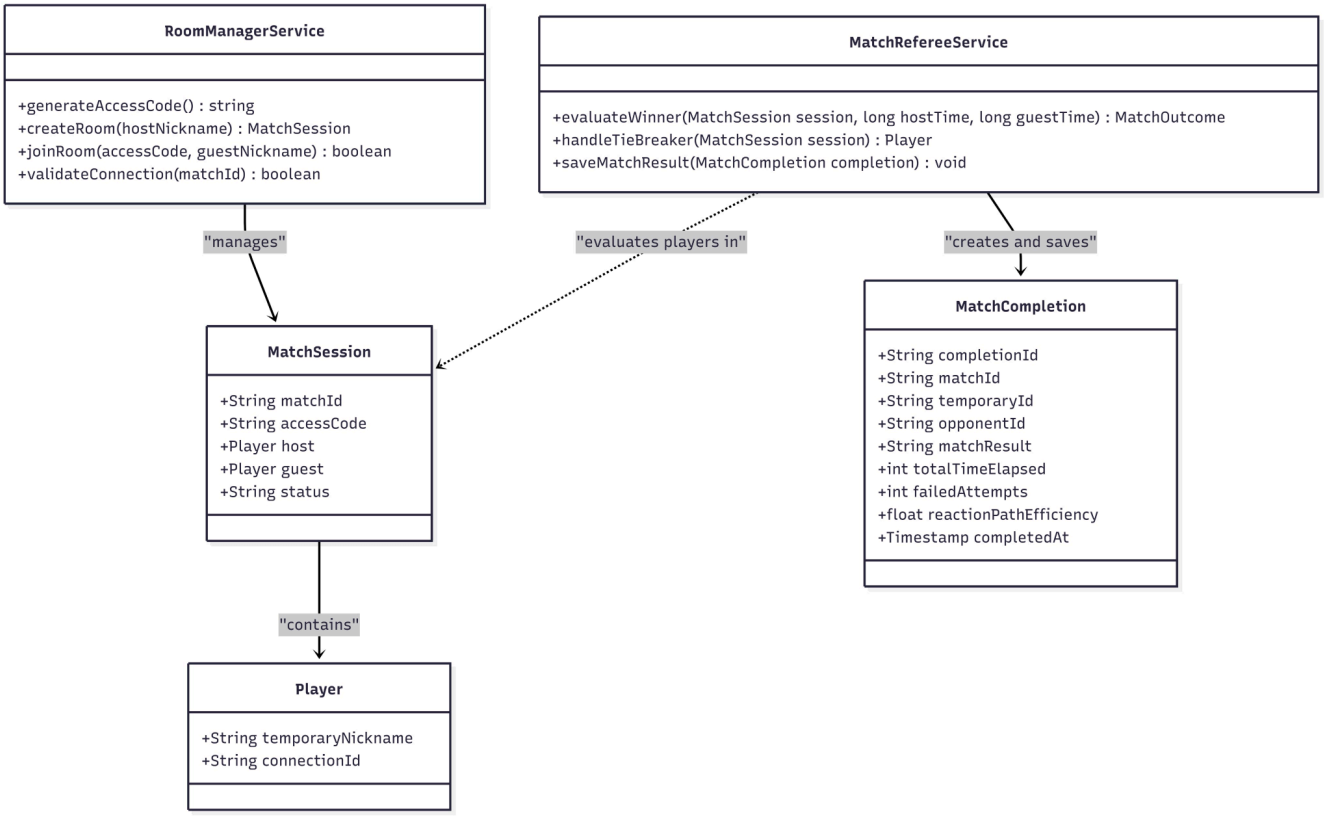
3.3 Real-Time Speed-to-Compound Evaluation

- User Interface Design

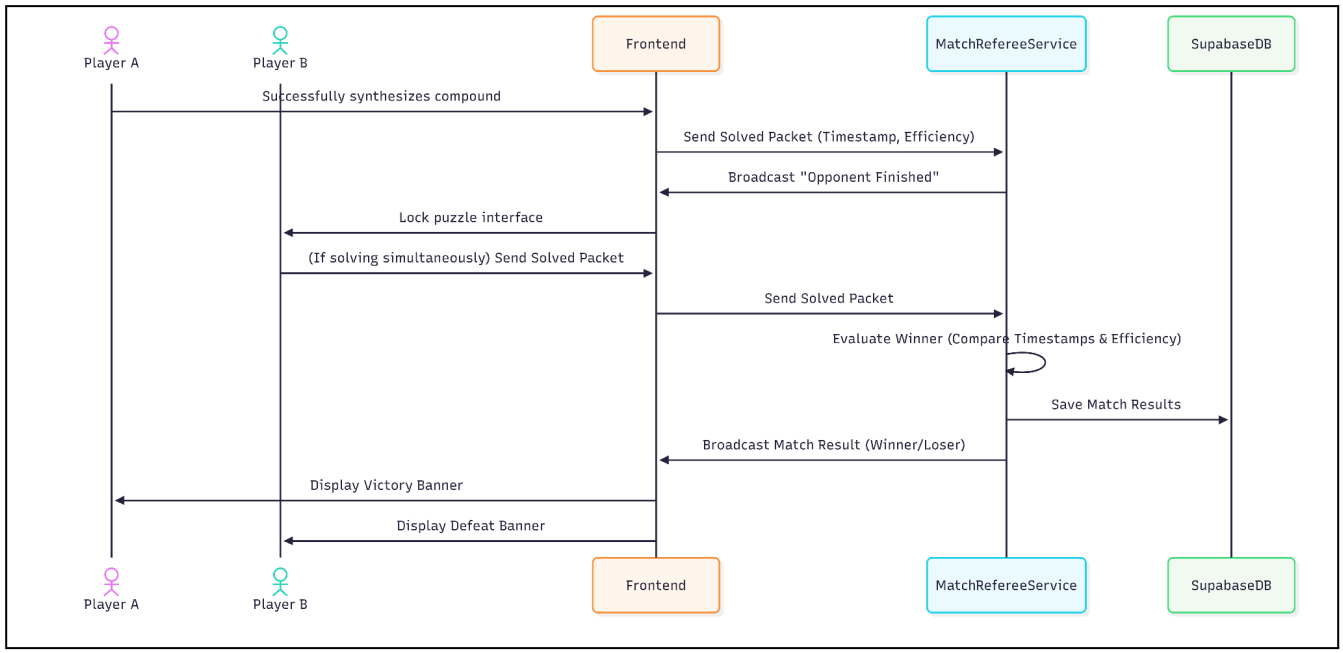


- Front-end component(s)
    - SpeedEvaluatorClient
        - Detects a valid compound synthesis and instantly transmits a 'solved' packet (including timestamp and efficiency) to the backend. Locks the interface upon receiving an opponent's victory signal.

        - React Hook / Context API
    - MatchResultBanner
        - Displays the outcome of the match (Win/Loss/Draw) based on the backend evaluation.
        - React UI Component
- Back-end component(s)
    - MatchRefereeService
        - Receives completion packets from both players. Compares timestamps and reaction path efficiency to determine the winner with 100% computational accuracy. Handles tie-breakers (failed attempts).
        - Spring Boot Service Class
- Object-Oriented Components
    - Class Diagram



    - Sequence Diagram



- Data Design
    - ERD or schema

MATCH_COMPLETION			
string	completionId	PK	Unique UUID for completion record
string	matchId	FK	Links to match lobby
string	temporaryId	FK	Links to SESSION_NICKNAME
string	opponentId		Competitor's nickname
string	matchResult		WIN, LOSS, or DRAW
int	totalTimeElapsed		Match duration in seconds
int	failedAttempts		Count of incorrect actions
float	reactionPathEfficiency		Calculated algorithmic score
timestamp	completedAt		Precise end time